Nominal Anti-Unification Modulo Equational Theories

Alexander Baumgartner

Instituto de Ciencias de la Ingeniería, Universidad de O'Higgins, Rancagua, Chile

Daniele Nantes-Sobrinho

Department of Mathematics, University of Brasília, Brazil
Department of Computing, Imperial College London, London, UK

Abstract

This paper investigates nominal anti-unification, i.e., the problem of computing least general generalizations of terms in the presence of binding structures and freshness constraints. We extend this problem to account for equational theories, specifically associativity (A), commutativity (C), and their combination (AC), yielding the notion of anti-unification modulo a theory E. Solutions in this setting, called E-generalizations, must consider both, the syntactic structure of nominal terms, and the algebraic properties imposed by E. In the general case, the problem has no solution due to the unbounded supply of atoms. To obtain a finitary setting, we fix a finite set of atoms A and introduce a rule-based algorithm for computing E-generalizations where only atoms from A are allowed to appear. The algorithm includes dedicated rules for reasoning modulo E and systematically explores permutations of subterms, which may lead to exponential blow-up in the number of generalizations. Although the computed set is not minimal, we describe heuristics that lead to computing promising E-generalizations in reasonable time. We also give an implementation of our algorithm which can be used to explore different optimization strategies that improve efficiency in practice, up to archiving tractability.

Keywords:
anti-unification, binding constructs, nominal techniques, equational theories

Email addresses: `alexander.baumgartner.x@gmail.com` (Alexander Baumgartner), `dnantess@imperial.ac.uk` (Daniele Nantes-Sobrinho)

1. Introduction

The anti-unification problem [1] is concerned with finding a generalization of two given input expressions in a formal language, e.g., finding a more general program between two different programs, or a more general first-order formula, or a more general proof. The idea of a generalization is to capture the common structure of the two input expressions. Interestingly, solving anti-unification problems plays a central role in the management of systems: it enables practitioners to identify and extract common functionalities in order to organize them in libraries of generic components that make systems more modular, maintainable, and expressive. Naturally, such functionalities may involve equational theories as demonstrated in the recent work [2].

Anti-unification problems modulo theories are concerned with finding an expression r that generalizes two given expressions s and t and preserves as much of their common structure as possible, while taking into account algebraic properties of operators – that is, their underlying equational theories. For instance, the expressions $s = a + g(b, h(c))$ and $t = g(h(a), h(c)) + a$ can be generalized by $r' = X + Y$. However, once the commutative property of the *addition* operator $+$ is taken into account, it becomes clear that r' does not represent the full structural similarity between s and t. A more informative generalization is $r = a + g(X', h(Y'))$, which retains more of the similarities between the two expressions by considering commutativity of $+$. While both r and r' generalize s and t modulo commutativity, r is less general than r'. We are interested in generalizations that cover as much of the common structure of the input expressions as possible, considering a given equational theory.

The anti-unification problem becomes significantly more complex in languages that feature binding constructs and equational theories, such as first-order logic, Milner's π-calculus, λ-calculus, and others. The key challenge includes proper handling of bound variable renaming (α-equivalence), and reasoning modulo other equivalences (e.g., β- and η-conversion). Most existing frameworks impose restrictions on the problem space to manage this complexity, tailoring their assumptions and capabilities to their specific goals. In particular, substantial work has been dedicated to solving anti-unification problems, with and without equational theories, in the setting of higher-order patterns, a restricted class of simply typed λ-terms where free variables can only be applied to a sequence of distinct bound variables [3, 4]. For a broader perspective on recent developments in anti-unification, we refer the reader to

the comprehensive survey by Cerna and Kutsia [5].

We are interested in solving anti-unification problems involving expressions with binders, such as first-order logic formulas with constraints. An example of such an anti-unification problem is

$$\begin{aligned} s &= \texttt{implies}(\forall x.(\texttt{P}(x) \wedge \varphi), \varphi), \quad \text{if } x \notin fv(\varphi), \\ t &= \texttt{implies}(\forall y.(\varphi \wedge \texttt{P}(y)), \varphi), \quad \text{if } y \notin fv(\varphi), \end{aligned} \tag{1}$$

where $\texttt{P}$ is a unary predicate, φ is a first-order formula, and $fv(\varphi)$ denotes the set of free variables in φ. The expression $r = \texttt{implies}(\forall z.(\texttt{P}(z) \wedge \varphi), \varphi)$ is a generalization of s and t modulo commutativity of $\wedge$ and renaming of the bound variables x and y.

An important detail is that x, y and φ are variables at different levels. The symbols x and y are *object-level* variables, representing elements of the domain in a logical interpretation, and are subject to binding. Whereas φ is a *meta-level* variable that represents an entire expression or formula, and it is not subject to binding. In this paper we follow the nominal approach, which is a general setting to deal with binding and freshness conditions. Nominal terms extends the first-order syntax with atoms $a, b, \ldots$, meta-variables $X, Y, \ldots$ (which will be simply called 'variables') and abstractions $a.t$ of atoms over terms. Intuitively, atoms represent object-level variables, that can be bound, and variables represent meta-level placeholders, which can be replaced (i.e., instantiated) by nominal terms. This distinction allows us to reason formally about bindings and instantiations, like illustrated in the example above.

The α-equivalence relation $\approx$ of nominal terms is defined up to swapping bound atoms, possibly under consideration of a freshness context, say ∇. A freshness context is a finite set of freshness constraints of the form $a\#X$ that expresses the fact that a is fresh for X, i.e., a cannot occur freely in instantiations of X. For example, $\forall a.a \approx \forall b.b$ because applying the permutation $(a\,b)$ in one side, produces (is equal to) the other side: $(a\,b) \cdot \forall a.a \equiv \forall b.b$. Here $\forall$ is unary function symbol in our signature. More interestingly, we can express renamings of bound names in terms that contain meta-variables, e.g., $\forall a.X \approx \forall b.X$ if both $a\#X$ and $b\#X$ hold.

In the nominal syntax with added equational reasoning, the expressions

in (1) and their freshess conditions can be specified[1] within the language as

$$\begin{aligned} &\{a\#X\} \vdash \texttt{implies}(\forall a.(\texttt{P}(a) \wedge X), X), \\ &\{b\#X\} \vdash \texttt{implies}(\forall b.(X \wedge \texttt{P}(b)), X), \end{aligned} \tag{2}$$

where $\wedge$ is a binary commutative symbol and $\texttt{implies}, \forall$ are unary function symbols of the given signature. The freshness constraints $a\#X$ and $b\#X$ appearing on the left of $\vdash$ implement the fact that a and b must not appear freely when instantiating X at the respective term on the right-hand side of $\vdash$. A generalization of the pairs in 2 is expressed within the nominal language as a pair $\langle \Delta, r \rangle$, called *term-in-context*, that denotes the fact that the term r generalizes both terms in the right of the $\vdash$ in (2) under consideration of the freshness context Δ, possibly modulo commutativity of $\wedge$. We will show how to compute this generalization later in Example 7.

A key subproblem in nominal anti-unification is the nominal *equivariance problem*, which arises when attempting to merge multiple anti-unification problems, such as $s_1 \triangleq t_1$ and $s_2 \triangleq t_2$, under a common freshness context Δ. Here, each pair $s_i \triangleq t_i$ represents a problem of computing a generalization between s_i and t_i. The equivariance problem consists of finding a permutation of atoms, π, such that $\pi \cdot s_1 \approx s_2$ and $\pi \cdot t_1 \approx t_2$, modulo the freshness constraints in Δ. When such a permutation exists, the two problems are equivalent up to renaming some atoms and can be merged. The current procedure to equivariance problems [7] cannot handle equational theories.

Contributions. We extend the nominal anti-unification algorithm introduced in [7] by proposing a sound and complete rule-based algorithm, $\texttt{NAU}_E$, that supports reasoning modulo equational theories for associativity (A), commutativity (C), and associativity-commutativity (AC). Generalizations that consider equational theories are named E-generalizations. To handle these theories, we introduce a new set of rules for solving equational equivariance problems, which determine whether a permutation π exists such that $\pi \cdot s_1 \approx_E s_2$ and $\pi \cdot t_1 \approx_E t_2$, where $\approx_E$ denotes equality modulo both, α-equivalence and the equational theory E. This generalizes the standard equivariance check to accommodate the algebraic structure of terms. We prove that the $\texttt{NAU}_E$ algorithm computes a finite set of E-generalizations, though not necessarily minimal. Further details are provided in §5.

[1] These are called *schemas* in [6].

Equational theories such as A and AC are known to cause branching during anti-unification. To address this, we extend the underlying algorithm with targeted optimizations inspired by [4]. In particular, we observe that restricting attention to *linear* least general generalizations, where each variable appears at most once. By doing so, we avoid the need to explicitly solve the equivariance problem, which simplifies reasoning. Notably, this restriction does not reduce the overall number of branches in the generalization computation, but it slightly improves tractability as it avoids the branching when solving equivariance problems.

We provide an implementation of $\texttt{NAU}_E$,[2] which incorporates the proposed optimizations and computes a finite and complete (though not necessarily minimal) set of E-generalizations. While the least general generalizations are included in the output, more general, potentially redundant, E-generalizations may also be present. The implementation strategy is detailed in §6. Several illustrative examples are provided in the paper and on the implementation website to showcase the algorithm's behavior in practice.

A preliminary version of this work appeared in the UNIF 2020 workshop [8].

Paper Organization. The structure of the paper is as follows. In §2, we introduce the necessary background on the nominal language that underpins the remainder of the work. Section §3 presents the simplification rules $\mathfrak{R}$ for anti-unification in the empty theory and lists some of its properties. In §4, we extend these rules with equational decomposition rules and examine their properties. Section §5 introduces the rules underlying the equivariance algorithm. In §6, we provide details of our implementation and outline our branching strategy. Section §7 contains comparisons with the current related work and §8 concludes the paper.

2. Preliminaries

We assume the reader to be familiar with the nominal syntax and only recall the main concepts and notations that are needed in the paper; for more details we refer the reader to [9, 10].

[2] Available at `https://nau-eq.web.app`

2.1. Nominal Syntax

Fix a countable infinite set of *variables* $\mathcal{X} = \{X, Y, Z, \ldots\}$ and a countable infinite set of *atoms* $\mathcal{A} = \{a, b, c, \ldots\}$, such that $\mathcal{X} \cap \mathcal{A} = \emptyset$. Variables represent meta-level *unknowns* and atoms *object-level* variable symbols. Atoms are identified by their name, so it will be redundant to say two atoms a and b are different. A signature Σ is a set of *term-formers* (disjoint from $\mathcal{A}$ and $\mathcal{X}$) such that to each $f \in \Sigma$ is assigned a unique non-negative integer n, called the *arity* of f , written as $f : n$.

A *permutation* π is a bijection $\mathcal{A} \to \mathcal{A}$ that is identity almost everywhere, i.e., the support $\mathsf{supp}(\pi) := \{a \in A \mid \pi(a) \neq a\}$ is finite. Id denotes the identity permutation and π^{-1} is the inverse of permutation π. The composition of two permutations π and π' will be denoted as $\pi \circ \pi'$. We may represent a permutation by a finite sequence of swappings. A *swapping* of two atoms a and b is written as $(a\,b)$. It is a convenient representation, since composition reduces to concatenating the given sequences of swappings. E.g., $(a\,b)(c\,d) \circ (b\,c) = (a\,b)(c\,d)(b\,c)$ and its inverse is $(b\,c)(c\,d)(a\,b)$.

Nominal terms are given by the grammar:

$$s, t ::= a \mid \ \pi \cdot X \mid \ a.t \mid \ f^E(t_1 \ldots t_n),$$

where a is an *atom*, $\pi \cdot X$ is a *suspension* (or meta-variable), $a.t$ denotes the *abstraction* of atom a in the term t, and $f^E(t_1, \ldots t_n)$ is a *function application*, where $f^E \in \Sigma$ and $f^E : n$. The parameter E describes the equational theory assigned to the function symbol f. It can be $\emptyset$, A, C or AC.

In the case of $f^\emptyset$ we omit the superscript, writing simply f. In an abstraction $a.t$ the atom a is *bound* in t. An atom a is *free* in a term t if it does not occur in the scope of an abstraction of a. $\mathsf{Vars}(t)$, $\mathsf{FA}(t)$ and $\mathsf{BA}(t)$ denote the set of variables, free and bound atoms of t, respectively. Moreover, $\mathsf{Atoms}(t)$ is the set of all atoms that occur in a term t. The *head* of a term s, denoted by $\mathsf{Head}(s)$, is defined as $\mathsf{Head}(a) = a$, $\mathsf{Head}(\pi \cdot X) = X$, $\mathsf{Head}(a.t) = .$ and $\mathsf{Head}(f^E(t_1 \ldots t_n)) = f^E$.

We write $(\overline{s})_m$ to denote the m-tuple of terms $(s_1, \ldots, s_m)$, where m is a non-negative integer. Given two non-negative integers $k \leq m$, we write $(\overline{s})_{k..m}$ to denote the tuple $(s_k, \ldots, s_m)$. We assume that associative function applications are *flattened*, writing, for instance $f^A(t_1, t_2, t_3)$ instead of $f^A(f^A(t_1, t_2), t_3)$ or $f^A(t_1, f^A(t_2, t_3))$. Since flattening is performed recursively, one might end up with $f^A(\overline{t})_m$ where $m \geq 2$ arbitrarily large, although functions that are associative and/or commutative are always of arity 2. For

convenience reasons, the notation is extended to $f^A(t)$ (f^A applied to a single argument) which stands for the term t.

The *action of a permutation* π on a term t, denoted $\pi \cdot t$, is defined by the structure of t as follows:

$$\pi \cdot a = \pi(a) \qquad \pi \cdot (a.t) = \pi(a).(\pi \cdot t)$$
$$\pi \cdot (\pi' \cdot X) = (\pi \circ \pi') \cdot X \qquad \pi \cdot f(t_1, \ldots, t_n) = f(\pi \cdot t_1, \ldots, \pi \cdot t_n)$$

The *difference set* of two permutations π and π' is defined as $\mathsf{ds}(\pi, \pi') = \{a \in \mathcal{A} \mid \pi \cdot a \neq \pi' \cdot a\}$. Intuitively, permutations are used to implement atom renaming.

Example 1. *To rename the bound atom a in the term $a.f(a, X)$ to b, the permutation $(a\,b)$ can be used, i.e., $(a\,b) \cdot (a.f(a, X)) = b.f(b, (a\,b) \cdot X)$. Note that $(a\,b) \cdot X$ denotes the fact that the renaming $(a\,b)$ is suspended in X. It can only take effect when a ground term (i.e., a term without meta-variables) is being assigned to X.*

Definition 1. Substitutions*, denoted by $\sigma, \gamma, \ldots$, are defined as maps from variables to nominal terms with finite domain. That is, they are of the form $\{X_1/t_1, \ldots, X_n/t_n\}$, where X_i/t_i is a pair of a variable X_i and a term t_i, $1 \leq i \leq n$. Such pairs are also called assignments.*

$\mathsf{Dom}(\sigma)$ and $\mathsf{Ran}(\sigma)$ denote the domain and the range of the substitution σ, respectively. We say that the variables in $\mathsf{Dom}(\sigma)$ are being *instantiated* by σ. The *action of a substitution* σ on a term t, denoted as $t\sigma$, is inductively defined by

$$a\sigma = a \qquad (a.t)\sigma = a.(t\sigma),$$
$$(\pi \cdot X)\sigma = \pi \cdot \sigma(X) \qquad f(t_1, \ldots, t_n)\sigma = f(t_1\sigma, \ldots, t_n\sigma),$$

where $\sigma(X)$ refers to carrying out the defined variable assignment, or, if X does not occur in $\mathsf{Dom}(\sigma)$, it is simply kept as is. As usual, substitution applications are written in postfix notation: $t(\sigma\gamma) = (t\sigma)\gamma$. Compositions of substitutions can be computed easily, i.e., given two substitutions σ and γ, the composition $\sigma\gamma$ is $\{X/t\gamma \mid X/t \in \sigma\} \cup \{X/t \in \gamma \mid X \notin \mathsf{Dom}(\sigma)\}$. The identity substitution is denoted by ε.

2.2. Equality and Freshness Constraints

In the nominal theory, there are two types of constraints:

- An α, E-*equality constraint* is a pair of the form $s \approx_E t$ and means that s is α-equivalent to t modulo E (sometimes we say that s and t are α, E-equivalent).

- A *freshness constraint* is a pair of the form $a\#t$ and its intended meaning is that if a occurs in t then it must be bound by an abstraction (equivalently, a cannot occur as a free atom in t).

Definition 2 (Freshness Context)**.** *A* primitive *freshness constraint is a pair of an atom and a variable of the form $a\#X$. A finite set of primitive freshness constraints is called a* freshness context*, and we use ∇ and Γ to denote them.*

Intuitively, $a\#X$ means that a cannot occur freely in any instance of X. Freshness constraints are used to avoid the problem of capturing free atoms. For convenience reasons, we also write $\{a_1, \ldots, a_n\#X\}$ to denote the freshness context $\{a_1\#X, \ldots, a_n\#X\}$, i.e., we may omit repeating the variable.

The notion of α, E-equivalence between two nominal terms and the freshness predicate $\#$ are defined by the rules in Figure 1 and Figure 2. In the case that there are no A, C or AC function symbols (i.e. $E = \emptyset$), the rules of Figure 2 do not apply, and the α, E-equivalence relation boils down to pure α-equivalence (denoted simply as $\approx$). Note that we will omit the f^A and f^{AC} from the right premise in the rules (`A`) and (`AC`), if $n = 2$. For instance, instead of $f^A(t_2) \approx_E f^A(s_2)$ we simply get $t_2 \approx_E s_2$. For more details, we refer the reader to [11].

A freshness constraint of the form $a\#a$ is called *inconsistent*, and we denote it as $\perp$. Given a set of constraints $S = \{a_1\#t_1, \ldots, a_n\#t_n\}$, the *normal form of* S, denoted by $\langle S\rangle_{nf}$, is the minimal set ∇ of freshness constraints obtained after exhaustively applying the rules of Figure 1 in a bottom-up manner. Given a freshness context ∇ and a substitution σ we define the *instance of* ∇ *under* σ as the set $\nabla\sigma = \{a\#X\sigma \mid a\#X \in \nabla\}$. A substitution σ *respects* a freshness context ∇ iff $\perp \notin \langle\nabla\sigma\rangle_{nf}$. From the construction above, it is easy to see that $\langle S\rangle_{nf} \subseteq \nabla$ iff $\nabla \vdash S$, where $\nabla \vdash S$ is an abbreviation of $\nabla \vdash a\#t$ for all $a\#t \in S$.

Example 2. *Consider the context $\nabla = \{a\#X, a\#Y\}$ and the substitution $\sigma = \{Y/f^A(b, X)\}$. It holds that σ respects ∇ as $\perp \notin \langle\nabla\sigma\rangle_{nf} = \{a\#X\}$.*

2.3. Nominal Anti-Unification Modulo E

In the nominal language, we can express terms that include structural constraints. For instance, consider the term $a.f(a, X)$, where it should be

$$\frac{}{\nabla \vdash a\#a'}\,(\texttt{\#at}) \qquad \frac{(\pi^{-1}\cdot a\#X)\in\nabla}{\nabla\vdash a\#\pi\cdot X}\,(\texttt{\#var}) \qquad \frac{}{\nabla\vdash a\#a.t}\,(\texttt{\#aba})$$

$$\frac{\nabla\vdash a\#t}{\nabla\vdash a\#a'.t}\,(\texttt{\#abb}) \qquad \frac{\nabla\vdash a\#t_1 \quad\cdots\quad \nabla\vdash a\#t_n}{\nabla\vdash a\#f(t_1,\ldots t_n)}\,(\texttt{\#f})$$

$$\frac{}{\nabla\vdash a\approx_E a}(\texttt{At}) \qquad \frac{\texttt{ds}(\pi,\pi')\#X\in\nabla}{\nabla\vdash \pi\cdot X\approx_E \pi'\cdot X}(\texttt{Var}) \qquad \frac{\nabla\vdash t\approx_E t'}{\nabla\vdash a.t\approx_E a.t'}(\texttt{Aba})$$

$$\frac{\nabla\vdash t\approx_E (a\,b)\cdot t' \qquad \nabla\vdash a\#t'}{\nabla\vdash a.t\approx_E b.t'}(\texttt{Abb}) \qquad \frac{\nabla\vdash t_1\approx_E t_1' \quad\cdots\quad \nabla\vdash t_n\approx_E t_n'}{\nabla\vdash f(t_1,\ldots t_n)\approx_E f(t_1',\ldots,t_n')}(\texttt{f})$$

Figure 1: Equality and Freshness Rules (see [12])

.

$$\frac{\nabla\vdash t_1\approx_E s_1 \quad \nabla\vdash f^A(t_2,\ldots,t_n)\approx_E f^A(s_2,\ldots,s_n)}{\nabla\vdash f^A(t_1,\ldots,t_n)\approx_E f^A(s_1,\ldots,s_n)}\,(\texttt{A})$$

$$\frac{\nabla\vdash t_1\approx_E s_i \quad \nabla\vdash t_2\approx_E s_{3-i}}{\nabla\vdash f^C(t_1,t_2)\approx_E f^C(s_1,s_2)}\,(\texttt{C})$$

$$\frac{\nabla\vdash t_1\approx_E s_k \quad \nabla\vdash f^{AC}(t_2,\ldots,t_n)\approx_E f^{AC}(s_1,\ldots,s_{k-1},s_{k+1},\ldots,s_n)}{\nabla\vdash f^{AC}(t_1,\ldots,t_n)\approx_E f^{AC}(s_1,\ldots,s_n)}\,(\texttt{AC})$$

Figure 2: Equational Rules (see [11]). Where $n\geq 2$, $1\leq k\leq n$ and $i\in\{1,2\}$.

avoided that the atom a occurs freely in instantiations of X. I.e., capturing of atoms during an instantiation of X should be avoided. This can be represented as the pair $\langle\{a\#X\}, a.f(a,X)\rangle$, where the freshness constraint $a\#X$ enforces the restriction on X. More generally, a *term-in-context* is a pair $\langle\nabla, t\rangle$ of a freshness context ∇ and a term t.

We now define the relation that captures when one term-in-context is more general than another. Since equational theories are involved, this relation compares equivalence classes of terms under a theory E.

Definition 3. *A term-in-context* $\langle\nabla_1, t_1\rangle$ *is* more general modulo E *than*

a term-in-context $\langle \nabla_2, t_2 \rangle$*, written* $\langle \nabla_1, t_1 \rangle \preceq_E \langle \nabla_2, t_2 \rangle$*, if there exists a substitution* σ *that respects* ∇_1 *such that* $\nabla_2 \vdash \nabla_1\sigma$ *and* $\nabla_2 \vdash t_1\sigma \approx_E t_2$*. Two terms-in-context* $\langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla_2, t_2 \rangle$ *are called* equi-general*, written* $\langle \nabla_1, t_1 \rangle \simeq_E \langle \nabla_2, t_2 \rangle$ *if* $\langle \nabla_1, t_1 \rangle \preceq_E \langle \nabla_2, t_2 \rangle$ *and* $\langle \nabla_2, t_2 \rangle \preceq_E \langle \nabla_1, t_1 \rangle$*. We write* $\prec_E$ *to denote the strict part of* $\preceq_E$*, that is,* $\langle \nabla_1, t_1 \rangle \prec_E \langle \nabla_2, t_2 \rangle$ *iff* $\langle \nabla_1, t_1 \rangle \preceq_E \langle \nabla_2, t_2 \rangle$ *but not* $\langle \nabla_2, t_2 \rangle \preceq_E \langle \nabla_1, t_1 \rangle$*.*

Example 3. *The term-in-context* $\langle \{a\#X, a\#Y\}, f^A(a, Y) \rangle$ *is more general than* $\langle \{a\#X\}, f^A(a, b, X) \rangle$ *modulo associativity. That is,* $\langle \{a\#X, a\#Y\}, f^A(a, Y) \rangle \preceq_E \langle \{a\#X\}, f^A(a, b, X) \rangle$*. We already verified in Example 2 that the substitution* $\sigma = \{Y/f^A(b, X)\}$ *respects* $\{a\#X, a\#Y\}$*. In addition to that, we have* $\{a\#X\} \vdash f^A(a, Y)\sigma \approx_E f^A(a, b, X)$*.*

The notion used to denote the set of atoms that occur in a term is extended to freshness contexts and sequences of terms or contexts. For instance, $\mathsf{Atoms}(\nabla)$ denotes the set of atoms that occur in the freshness contexts ∇ and $\mathsf{Atoms}(t, s, \nabla)$ is the set $\mathsf{Atoms}(t) \cup \mathsf{Atoms}(s) \cup \mathsf{Atoms}(\nabla)$, where t, s are terms.

Based on the relation $\preceq_E$, the notion of a generalization of two given terms-in-context modulo an equational theory E is being introduced. Intuitively, a generalization modulo E represents the common structure of the given terms while preserving their constraints. It is a term-in-context that is more general than both of the given original terms-in-context. I.e., both original terms can be obtained by instantiation (via two different substitutions), taking into account the theory E and the freshness context. We will refer to a generalization modulo E as an *E-generalization* when $E \neq \emptyset$, and just as a *generalization* when $E = \emptyset$.

Definition 4 (E-generalization)**.** *The pair* $\langle \nabla, r \rangle$ *is called an* E-generalization *of* $\langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla_2, t_2 \rangle$ *if it is more general modulo* E *than each of them, i.e.,* $\langle \nabla, r \rangle \preceq_E \langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla, r \rangle \preceq_E \langle \nabla_2, t_2 \rangle$*. It is called a* least general E-generalization *(for short,* $\mathtt{lgg}_E$*) of* $\langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla_2, t_2 \rangle$ *if there is no other* E*-generalization, say* $\langle \Delta, r' \rangle$*, of* $\langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla_2, t_2 \rangle$ *which satisfies* $\langle \nabla, r \rangle \prec_E \langle \Delta, r' \rangle$*. The set of all* E*-generalizations of* $\langle \nabla_1, t_1 \rangle$ *and* $\langle \nabla_2, t_2 \rangle$ *is denoted as* $\mathcal{G}_E(\langle \nabla_1, t_1 \rangle, \langle \nabla_2, t_2 \rangle)$*.*

Example 4. $\langle \{a\#X, a\#Y\}, f^A(a, Y) \rangle$ *is an* E*-generalization of the terms-in-context* $\langle \{a\#X\}, f^A(a, b, X) \rangle$ *and* $\langle \emptyset, f^A(a, c, d) \rangle$*. It remains to check that* $\langle \{a\#X, a\#Y\}, f^A(a, Y) \rangle \preceq_E \langle \emptyset, f^A(a, c, d) \rangle$ *holds. Consider the substitution*

$\gamma = \{X/c, Y/f^A(c,d)\}$. *Note that* γ *respects* $\{a\#X, a\#Y\}$ *and, in addition,* $\{a\#X\} \vdash f^A(a,Y)\gamma \approx_E f^A(a,c,d)$. *Note that* $\langle\{a\#X\}, Z\rangle$ *is also an* E*-generalization of the same two terms-in-context, but it is not an* $\mathtt{lgg}_E$ *because* $\langle\{a\#X\}, Z\rangle \prec_E \langle\{a\#X, a\#Y\}, f^A(a,Y)\rangle$. *Take* $\sigma' = \{Z/f^A(a,Y)\}$. *It respects* $\{a\#X\}$ *and* $\{a\#X, a\#Y\} \vdash Z\sigma' \approx_E f^A(a,Y)$.

From now on, to simplify the notation and avoid clutter, we only consider E-generalizations of two terms-in-context with the same freshness context, i.e., the input terms-in-context are of the form $\langle\nabla, t_1\rangle$ and $\langle\nabla, t_2\rangle$. The notation for the set of all E-generalizations of $\langle\nabla, t_1\rangle$ and $\langle\nabla, t_2\rangle$ simplifies to $\mathcal{G}_E(\nabla, t_1, t_2)$.

Definition 5 (Minimal Complete Set of E-generalizations)**.** *The* minimal complete set of E-generalizations *of the terms-in-context* $\langle\nabla, t_1\rangle$ *and* $\langle\nabla, t_2\rangle$, *denoted as* $\mathtt{mcsg}_E(\nabla, t_1, t_2)$, *is a subset of* $\mathcal{G}_E(\nabla, t_1, t_2)$ *satisfying:*

- *(Completeness) For each* $\langle\Delta, r\rangle \in \mathcal{G}_E(\nabla, t_1, t_2)$ *there exists* $\langle\nabla', r'\rangle \in \mathtt{mcsg}_E(\nabla, t_1, t_2)$ *such that* $\langle\Delta, r\rangle \preceq_E \langle\nabla', r'\rangle$.
- *(Minimality) If* $\langle\Delta, r\rangle, \langle\nabla', r'\rangle \in \mathtt{mcsg}_E(\nabla, t_1, t_2)$ *and* $\langle\Delta, r\rangle \preceq_E \langle\nabla', r'\rangle$ *then* $\langle\Delta, r\rangle = \langle\nabla', r'\rangle$.

The minimal complete set of E-generalizations defines the generalization type of an equational theory E, which may be one of the following:

- It is *unitary* if $\mathtt{mcsg}_E(\nabla, s, t)$ exists for all terms-in-context $\langle\nabla, s\rangle$ and $\langle\nabla, t\rangle$, and it is always a singleton.
- It is *finitary*, if $\mathtt{mcsg}_E(\nabla, s, t)$ exists and is finite, for all terms-in-context $\langle\nabla, s\rangle$ and $\langle\nabla, t\rangle$. Moreover, there exist $\langle\nabla', s'\rangle$ and $\langle\nabla', t'\rangle$ such that $\mathtt{mcsg}_E(\nabla', s', t')$ is not a singleton.
- It is *infinitary*, if $\mathtt{mcsg}_E(\nabla, s, t)$ exists for all terms-in-context $\langle\nabla, s\rangle$ and $\langle\nabla, t\rangle$, and there exist $\langle\nabla', s'\rangle$ and $\langle\nabla', t'\rangle$ such that $\mathtt{mcsg}_E(\nabla', s', t')$ is infinite.
- It is *nullary*, if for some $\langle\nabla, s\rangle$ and $\langle\nabla, t\rangle$, the set $\mathtt{mcsg}_E(\nabla, s, t)$ does not exist.

Remark 1 (Extension from [7])**.** *Note that, if we have an infinite number of atoms in the language, the relation* $\prec_E$ *is not well-founded:*

$$\langle\emptyset, X\rangle \prec_E \langle\{a\#X\}, X\rangle \prec_E \langle\{a\#X, b\#X\}, X\rangle \prec_E \ldots$$

As a consequence, two terms-in-context may not have an $\mathtt{lgg}_E$ *and the minimal complete set of E-generalizations does not exist. Therefore, our constructions will be based on a finite set of atoms that can be used in E-generalizations, like suggested in [7] for the empty theory.*

A term t is *based on a set of atoms* A (abbreviated as A-based) iff $\mathsf{Atoms}(t) \subseteq A$. Similarly, a freshness context ∇ is A-based iff $\mathsf{Atoms}(\nabla) \subseteq A$, and a term-in-context $\langle \nabla, t \rangle$ is A-based iff $\mathsf{Atoms}(\nabla) \cup \mathsf{Atoms}(t) \subseteq A$. A permutation π is A-based iff $\mathsf{supp}(\pi) \subseteq A$.

All the other notions can be adapted in a straightforward manner. Just to mention a few: An E-generalizations of two (possibly A-based) terms-in-contexts $\langle \nabla, t \rangle$ and $\langle \nabla, s \rangle$ might be A-based. $\mathcal{G}_E(\nabla, t, s)$ being A-based denotes the complete set of A-based E-generalizations. That is, A-based $\mathcal{G}_E(\nabla, t, s)$ is a (possibly strict) subset of $\mathcal{G}_E(\nabla, t, s)$ modulo the theory. A-based $\mathtt{mcsg}_E(\nabla, t, s)$ means the minimal complete set of A-based E-generalizations, and an A-based $\mathtt{lgg}_E$ is an element of that set.

Nominal anti-unification problem modulo E. We are interested in computing E-generalizations of two terms-in-context, restricted to a finite set of atoms A. This problem is called the nominal *anti-unification problem modulo E* based on A. It is formally specified as follows:

Given: Two A-based terms-in-context $\langle \nabla, t \rangle$ and $\langle \nabla, s \rangle$.

Find: An A-based $\mathtt{lgg}_E$ of $\langle \Delta, t \rangle$ and $\langle \nabla, s \rangle$.

Since the set of atoms A is a fixed parameter, it is crucial to know how the selection of A affects the A-based $\mathtt{mcsg}_E(\nabla, t, s)$ of the given freshness-context ∇ and terms t, s. In [7] it has been shown that the amount of atoms that should be available in A is closely related to the number of abstractions that occur in either t or s.

Theorem 1. *Given two terms-in-context* $\langle \nabla, t \rangle$ *and* $\langle \nabla, s \rangle$*. Let* n, m *be the numbers of abstraction occurrences in* t, s*, respectively. Let* $A_1 = \mathsf{Atoms}(t, s, \nabla)$ *and* A_2 *be a set of* $\min(n, m)$ *atoms such that* $A_1 \cap A_2 = \emptyset$*. Let* $A = A_1 \cup A_2$ *and* $B \supseteq A$ *be a finite set of atoms.*

For any B-based $\mathtt{lgg}_E$ $\langle \Gamma, r \rangle$ *of* $\langle \nabla, t \rangle$ *and* $\langle \nabla, s \rangle$ *there is an A-based* $\mathtt{lgg}_E$ $\langle \Gamma', r' \rangle$ *of* $\langle \nabla, t \rangle$ *and* $\langle \nabla, s \rangle$ *such that* $\Gamma \vdash r \simeq_E r'$*.*

Although Theorem 1 has been restated to fit into the setting with equational theories, adjusting the proof given in [7] is straightforward. We say

that A is *saturated* for ∇, t and s if $\mathsf{Atoms}(t, s, \nabla) \subseteq A$ (i.e., t, s, and ∇ are A-based) and there are at least k additional atoms in A, where k is the minimum number of abstraction occurrences in t and abstraction occurrences in s. Note that, having more atoms than necessary in A leads to cluttering the freshness contexts in A-based $\mathtt{mcsg}_E(\nabla, t, s)$. For that reason, a natural choice is to take the minimal saturated set of atoms.

Example 5. *Consider the terms-in-context* $\langle \emptyset, a.f^C(a, b)\rangle$ *and* $\langle \emptyset, b.f^C(a, b)\rangle$. *Their* $\{a, b\}$*-based* $\mathtt{lgg}_E$ *is* $\langle \emptyset, X\rangle$ *because* α*-equivalent renaming of the bound atoms can't be done. On the other hand, the* $\{a, b, c\}$*-based* $\mathtt{lgg}_E$ *is* $\langle \{c\#X\}, c.f^C(c, X)\rangle$. *Adding yet an other atom* d*, we get* $\langle \{c\#X, d\#X\}, c.f^C(c, X)\rangle$ *as their* $\{a, b, c, d\}$*-based* $\mathtt{lgg}_E$.

To initiate the anti-unification process, we construct an anti-unification triple (AUT_E) of the form $X : t \triangleq s$, where X is a fresh variable – called *generalization variable* – that does neither appear in the terms s, t nor in the freshness context ∇. The goal is to find A-based generalizations of $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$ where A is saturated for t, s, and ∇.

In the following sections, we present an algorithm, denoted $\mathtt{NAU}_E$, that computes a complete set of such E-generalizations. The algorithm is exhaustive: it explores all possibilities arising from function symbols governed by associativity (A), commutativity (C), or both (AC). As a result, the computed set is not guaranteed to be minimal and may contain repeated generalizations arising from different branches of the search space. Minimality can be archived in a subsequent step, utilizing an equational matching algorithm.

Remark 2. *It is worth noting that all definitions presented in this section can be naturally restricted to the case* $E = \emptyset$*, corresponding to the purely syntactic setting originally introduced in [7]. In this special case, we simply replace the relations* $\approx_E$ *and* $\preceq_E$ *with their counterparts* $\approx$ *and* $\preceq$*, respectively, and disregard the equational reasoning rules shown in Figure 2.*

3. Nominal Anti-Unification in the Empty Theory

In this section, we present the nominal equational anti-unification algorithm $\mathtt{NAU}_\emptyset$ for the empty theory. The algorithm is formulated as a rule-based system using the set of rules $\mathfrak{R}$ originally introduced in [7]. These rules compute nominal least general generalizations (lggs) of the input problem in the special case where $E = \emptyset$.

3.1. Rules of the algorithm $\mathtt{NAU}_\emptyset$

The rules in $\mathfrak{R}$ operate on tuples of the form $P;\ S;\ \Gamma;\ \sigma$, called *states*, and two global parameters A and ∇, where: P denotes the set of unsolved anti-unification triples $\{X_1 : t_1 \triangleq s_1, \ldots, X_m : t_m \triangleq s_m\}$ (called AUTs) that are yet to be solved; the store S denotes the set of solved triples; Γ denotes the freshness context computed so far; σ is a substitution that holds the generalized term; A is a finite set of atoms (the ones that are allowed in the generalization); ∇ is a freshness context. The symbol $\uplus$ denotes the disjoint union operation. We now describe each rule in detail.

(`Dec`) **Decomposition Rule**

$$\{X : \mathsf{h}(\bar{t})_m \triangleq \mathsf{h}(\bar{s})_m\} \uplus P; S; \Gamma; \sigma \Longrightarrow \bigcup_{i=1}^{m} \{Y_i : t_i \triangleq s_i\} \cup P; S; \Gamma; \sigma\{X/\mathsf{h}(\overline{Y})_m\},$$

where $\mathsf{h} \in \Sigma \cup A, m \geq 0$ and $Y_1, \ldots, Y_m$ are new variables.

Description: If both sides of the triple begin with the same function symbol of arity $m \geq 0$ or they are the same atom ($m = 0$), then the triple is decomposed component-wise into m new triples. The fresh generalization variables $Y_1, \ldots, Y_m$ of the newly created triples capture the pointwise structure of the original terms. Those variables are used in the term $\mathsf{h}(\overline{Y})_m$ that is a generalization of $\mathsf{h}(\bar{t})_m$ and $\mathsf{h}(\bar{s})_m$. The generalization variable X represents either $\mathsf{h}(\bar{t})_m$ or $\mathsf{h}(\bar{s})_m$. Therefore, it is being updated in σ, which holds the so-far computed generalization, by $X/\mathsf{h}(\overline{Y})_m$.

(`Abs`) **Abstraction Rule**

$$\{X : a.t \triangleq b.s\} \uplus P; S; \Gamma; \sigma \Longrightarrow \{Y : (c\,a) \cdot t \triangleq (c\,b) \cdot s\} \cup P; S; \Gamma; \sigma\{X/c.Y\},$$

where $c \in A$, Y is a new variable, $\nabla \vdash c\#a.t$ and $\nabla \vdash c\#b.s$.

Description: When both terms are abstractions, the algorithm applies a capture-avoiding renaming, using an atom $c \in A$ that is fresh in both, $a.t$ and $b.s$, considering ∇. The generalization is then extended with an abstraction over c. Note that the rule is only applicable if such an atom c exists in A. If no such atom is available, the rule cannot be applied. This can only happen if A is not saturated for the two input terms-in-context.

(`Sol`) **Solve Rule**

$$\{X : t \triangleq s\} \uplus P; S; \Gamma;\ \sigma \Longrightarrow P; S \cup \{X : t \triangleq s\}; \Gamma \cup \Gamma'; \sigma,$$

if none of the previous rules is applicable. $\Gamma' := \{a\#X \mid a \in A, \nabla \vdash a\#t \wedge \nabla \vdash a\#s\}$.

Description: If no other rule applies, the current AUT is considered a residual difference and is moved from P to S. A freshness context Γ' is computed to ensure that X is fresh for all atoms $a \in A$ that are fresh for both t and s under ∇. The solve rule captures dissimilarities between terms, representing them via variables.

(`Mer`) **Merge Rule**

$$\begin{aligned} &P; \{X : t_1 \triangleq s_1, Y : t_2 \triangleq s_2\} \uplus S; \Gamma; \sigma \\ &\qquad\qquad \Longrightarrow P; \{X : t_1 \triangleq s_1\} \cup S; \Gamma\{Y/\pi{\cdot}X\}; \sigma\{Y/\pi{\cdot}X\}, \end{aligned}$$

where π is an A-based permutation so that $\nabla \vdash \pi{\cdot}t_1 \approx t_2$ and $\nabla \vdash \pi{\cdot}s_1 \approx s_2$.

Description: This rule merges two solved triples when their respective components are equivalent up to an A-based permutation π. That is, t_1 and t_2, as well as s_1 and s_2, are α-equivalent under π. In that case, the variable X that represents t_1 and s_1 can be reused to also represent t_2 and s_2 by using $\pi \cdot X$. Therefore, Y is replaced by $\pi \cdot X$ in both, the freshness context and the substitution. The task of finding such a permutation π is known as the *equivariance problem*. We present an algorithm for solving equivariance problems and computing the corresponding permutation in Section 5.

3.2. Generalization computation with $\mathtt{NAU}_\emptyset$

To compute an A-based generalization of two given A-based terms-in-context $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$, the algorithm starts with the *initial state* $\{X : t \triangleq s\};\ \emptyset;\ \emptyset;\ \varepsilon$, where X is a fresh generalization variable not occurring in ∇, t, or s. The variable X intuitively represents the generalization term, which becomes progressively more specific as rules are applied. Starting from the initial state, the algorithm exhaustively applies the rules from $\mathfrak{R}$ in a non-deterministic manner.

The process results in a *final state*, a tuple of the form $\emptyset;\ S;\ \Gamma;\ \sigma$, where no more rule is applicable. In the final state, the set P of unsolved AUTs is

empty, S holds the residual differences between t and s, Γ is the accumulated freshness context, and σ is the computed substitution. The term-in-context $\langle \Gamma, X\sigma \rangle$ represents an A-based generalization (modulo the empty theory) of the input terms-in-context $\langle \nabla, t \rangle$ and $\langle \nabla, s \rangle$, capturing their common structure. All the differences of $\langle \nabla, t \rangle$ and $\langle \nabla, s \rangle$ are recorded in S.

We write $P;\ S;\ \Gamma;\ \sigma \Longrightarrow_{(\mathtt{R})} P';\ S';\ \Gamma';\ \sigma'$ to denote an application of a rule $(\mathtt{R}) \in \mathfrak{R}$ to the state $P;\ S;\ \Gamma;\ \sigma$ that results in $P';\ S';\ \Gamma';\ \sigma'$. Moreover, $\Longrightarrow^{n}_{(\mathtt{R})}$ denotes n consecutive applications of the rule $(\mathtt{R})$. $\overset{+}{\Longrightarrow}_{\mathfrak{R}}$ and $\overset{*}{\Longrightarrow}_{\mathfrak{R}}$ denote the transitive and reflexive-transitive closure of arbitrary rule applications.

In the following examples, we only show one assignment X/t of the computed substitution, namely the one that is used to obtain the generalization. To avoid clutter, all the other assignments are being omitted.

Example 6. *Let $\nabla = \emptyset$ and $A = \{a, b, c, d, a', b'\}$ be the global parameters. Consider the terms*

$$t = a.b.f(h_1(g(a), b), h_1(g(b), a)) \text{ and } s = c.d.f(h_2(d, g(c)), h_2(c, g(d)))$$

to be generalized. $\mathtt{NAU}_{\emptyset}$ *performs the following transformations:*

$$\begin{aligned}
&\{X : a.b.f(h_1(g(a), b), h_1(g(b), a)) \triangleq c.d.f(h_2(d, g(c)), h_2(c, g(d)))\}; \emptyset; \emptyset; \varepsilon \\
&\Longrightarrow^{2}_{(\mathtt{Abs})} \\
&\{Y : f(h_1(g(a'), b'), h_1(g(b'), a')) \triangleq f(h_2(b', g(a')), h_2(a', g(b')))\}; \emptyset; \emptyset; \sigma
\end{aligned}$$

where $\sigma = \{X/a'.b'.Y\}$ and Y is fresh variables. Next, we apply the decomposition rule and obtain:

$$\begin{aligned}
&\Longrightarrow_{(\mathtt{Dec})} \\
&\{Y_1 : h_1(g(a'), b') \triangleq h_2(b', g(a')), Y_2 : h_1(g(b'), a') \triangleq h_2(a', g(b'))\}; \emptyset; \emptyset; \sigma_1
\end{aligned}$$

where $\sigma_1 = \{X/a'.b'.f(Y_1, Y_2)\}$ and Y_1, Y_2 are fresh variables. Next, we apply the solve rule twice and obtain:

$$\begin{aligned}
&\Longrightarrow^{2}_{(\mathtt{Sol})} \\
&\emptyset; \{Y_1 : h_1(g(a'), b') \triangleq h_2(b', g(a')), Y_2 : h_1(g(b'), a') \triangleq h_2(a', g(b'))\}; \Gamma; \sigma_1
\end{aligned}$$

where $\Gamma = \{a, b, c, d\#Y_1,\ a, b, c, d\#Y_2\}$. Finally, we apply the merge rule:

$$\Longrightarrow_{(\mathtt{Mer})} \emptyset; \{Y_1 : h_1(g(a'), b') \triangleq h_2(b', g(a'))\}; \Gamma'; \{X/a'.b'.f(Y_1, (a'\,b') \cdot Y_1)\}$$

where $\Gamma' = \{a, b, c, d\#Y_1\}$*. In the merge step, the* A*-based permutation* $(a'\,b')$ *is used, since it hold that*

$$(a'\,b') \cdot h_1(g(a'), b') \approx h_1(g(b'), a') \text{ and } (a'\,b') \cdot h_2(b', g(a')) \approx h_1(a', g(b')).$$

Let $\sigma = \{X/a'.b'.f(Y_1, (a'\,b') \cdot Y_1)\}$ *and* $r = X\sigma$*. The computed generalization is* $\langle \Gamma', r \rangle$*. To recover the original inputs, the substitutions* $\sigma_1 = \{Y_1/h_1(g(a'), b')\}$ *and* $\sigma_2 = \{Y_2/h_2(b', g(a'))\}$ *can be obtained from the store. Applying them to* r *yields* $t = r\sigma_1$ *and* $s = r\sigma_2$*. As* σ_1 *and* σ_2 *respect* Γ'*, it follows that* $\langle \Gamma', r \rangle$ *is indeed a generalization of* $\langle \emptyset, t \rangle$ *and* $\langle \emptyset, s \rangle$*.*

We will build on the algorithm $\mathtt{NAU}_\emptyset$ to define an anti-unification procedure $\mathtt{NAU}_E$ that operates modulo an equational theory E (see Section 4). To keep the presentation self-contained, we recall key properties of $\mathtt{NAU}_\emptyset$, originally presented in [7]. We only repeat some interesting parts of the soundness proof, since it is the most crucial one when it comes to proving correctness of the extended algorithm $\mathtt{NAU}_E$. The other proofs can be found in the original work and the corresponding technical report [13].

Theorem 2. $\mathtt{NAU}_\emptyset$ *is terminating.*

The following lemma is a key ingredient in the soundness proof of $\mathtt{NAU}_\emptyset$. It ensures that the algorithm correctly accumulates all freshness constraints required to validate the computed generalizations. In particular, it guarantees that, whenever a freshness constraint of the form $c\#Z_k$ appears in the accumulated context, then the atom c is indeed fresh for both sides of the associated solved triple involving Z_k.

Lemma 1. *Let* $P_0; S_0; \Gamma_0; \vartheta_0 \overset{+}{\Longrightarrow}_{\mathfrak{R}} \emptyset; S_n; \Gamma_n; \vartheta_0\vartheta_1 \cdots \vartheta_n$ *be a derivation in* $\mathtt{NAU}_\emptyset$ *(with* ∇ *and* A*). The following holds for all* $0 \leq k \leq n$*: If* $Z_k : t_k \triangleq s_k \in S_k$ *and* $c\#Z_k \in \Gamma_k$ *for some* $c \in A$*, then* $\nabla \vdash c\#t_k$ *and* $\nabla \vdash c\#s_k$*.*

Proof. Proceed by induction on k. In the base case, $k = 0$, and the result follows from the assumption on $P_0; S_0, \Gamma_0; \vartheta_0$. For the inductive step, assume the result is true for k. Now we show it holds for $k + 1$.

Take $Z_{k+1} : t_{k+1} \triangleq s_{k+1} \in S_{k+1}$. We have two alternatives: Either $Z_{k+1} : t_{k+1} \triangleq s_{k+1}$ has been a subject of the (`Mer`) rule at this step, or not. If not, then either it was already in S_k or was introduced at this step. In either case, by IH or because it has been introduced with (`Sol`) rule, if $c\#Z_k \in \Gamma_k$ for a $c \in A$, then $\nabla \vdash c\#t_k$ and $\nabla \vdash c\#s_k$. If $Z_{k+1} : t_{k+1} \triangleq s_{k+1}$ was a

subject of the (Mer) rule, then there exists some $U_k : r_k \triangleq q_k \in S_k$, such that $\nabla \vdash \pi_k \cdot t_{k+1} \approx r_k$, $\nabla \vdash \pi_k \cdot s_{k+1} \approx q_k$. Moreover, for all $d\#U_k \in S_k$ we now have $\pi_k^{-1} \cdot d\#Z_{k+1} \in S_{k+1}$, and all $c\#Z_{k+1} \in S_k$ are retained in S_{k+1}. For these c's, since $Z_{k+1} : t_{k+1} \triangleq s_{k+1} \in S_k$, by the IH we have $\nabla \vdash c\#t_{k+1}$ and $\nabla \vdash c\#s_{k+1}$. As for $\pi_k^{-1} \cdot d\#Z_{k+1} \in S_{k+1}$, it remains to show $\nabla \vdash \pi_k^{-1} \cdot d\#t_{k+1}$ and $\nabla \vdash \pi_k^{-1} \cdot d\#s_{k+1}$. By the IH, it follows that $\nabla \vdash d\#r_k$. Then $\nabla \vdash \pi_k^{-1} \cdot d\#\pi_k^{-1} \cdot r_k$ and since $\nabla \vdash \pi_k \cdot t_{k+1} \approx r_k$, we get $\nabla \vdash \pi_k^{-1} \cdot d\#t_{k+1}$. Similarly, $\nabla \vdash \pi_k^{-1} \cdot d\#s_{k+1}$ holds using $\nabla \vdash d\#q_k$. $\square$

Soundness guarantees that $\mathtt{NAU}_\emptyset$ computes an A-based generalization of any two A-based input terms-in-context $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$.

Theorem 3 (Soundness of $\mathtt{NAU}_\emptyset$)**.** *Given terms t and s and a freshness context ∇, all based on a finite set of atoms A, if*

$$\{X : t \triangleq s\};\ \emptyset;\ \emptyset;\ \varepsilon \overset{+}{\Longrightarrow}_{\mathfrak{R}} \emptyset;\ S; \Gamma;\ \sigma$$

is a derivation obtained by an execution of $\mathtt{NAU}_\emptyset$, then $\langle \Gamma, X\sigma\rangle$ is an A-based generalization of $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$.

Proof. We prove a more general statement using well-founded induction on the length of derivations. Assume that $P_0; S_0; \Gamma_0; \vartheta_0 \overset{+}{\Longrightarrow}_{\mathfrak{R}} \emptyset; S_n; \Gamma_n; \vartheta_0\vartheta_1 \cdots \vartheta_n$ is a derivation in $\mathtt{NAU}_\emptyset$ (with ∇ and A) with the property (P): If $Z_0 : t_0 \triangleq s_0 \in S_0$, then for all $a \in A$ holds $a\#Z_0 \in \Gamma_0$ iff $\nabla \vdash a\#t_0$ and $\nabla \vdash a\#s_0$. Then, for any $Z_0 : t_0 \triangleq s_0 \in P_0 \cup S_0$ the following hold:

$$\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1 \cdots \vartheta_n\rangle \preceq \langle \nabla, t_0\rangle \text{ and } \langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1 \cdots \vartheta_n\rangle \preceq \langle \nabla, s_0\rangle.$$

(Induction Hypothesis) The statement is true for any derivation of the length $l < n$. We will prove that it holds for a derivation of length n:

$$P_0; S_0; \Gamma_0; \vartheta_0 \Longrightarrow^n_{\mathfrak{R}} \emptyset; S_n; \Gamma_n; \vartheta_0\vartheta_1 \cdots \vartheta_n.$$

Below the composition $\vartheta_i\vartheta_{i+1} \cdots \vartheta_k$ is abbreviated as ϑ_i^k with $k \geq i$. In addition, we denote as D the derivation $P_1;\ S_1; \Gamma_1; \vartheta_0\vartheta_1 \Longrightarrow^{n-1}_{\mathfrak{R}} P_n;\ S_n; \Gamma_n; \vartheta_0^n$. Let $Z_0 : t_0 \triangleq s_0$ be an AUT from $P_0 \cup S_0$ selected for the first step transformation. We only consider the interesting rules from $\mathfrak{R}$:

- The rule applied in the first step is (Abs):

Then $Z_0 = X$, $t_0 = a.t$, $s_0 = b.s$, $P_0 = \{X : a.t \triangleq b.s\} \uplus P'_0$, $S_1 = S_0$, $\Gamma_1 = \Gamma_0$, and $\vartheta_1 = \{X/c.Y\}$, where $\nabla \vdash c\#a.t$ and $\nabla \vdash c\#b.s$. The first step is $\{X : a.t \triangleq b.s\} \uplus P'_0; S_0; \Gamma_0; \vartheta_0 \Longrightarrow_{(\texttt{Abs})} P_1; S_1; \Gamma_1; \vartheta_0\vartheta_1$, and $P_1 = \{Y : (c\,a) \cdot t \triangleq (c\,b) \cdot s\} \cup P'_0$. By applying the IH in D, both $\langle \Gamma_n \setminus \Gamma_1, Y\vartheta_2^n\rangle \preceq \langle \nabla, (c\,a) \cdot t\rangle$ and $\langle \Gamma_n \setminus \Gamma_1, Y\vartheta_2^n\rangle \preceq \langle \nabla, (c\,b) \cdot s\rangle$ hold. Hence, there exist σ and φ that respect $\Gamma_n \setminus \Gamma_1$ such that: $\nabla \vdash (\Gamma_n \setminus \Gamma_1)\varphi$ and $\nabla \vdash (\Gamma_n \setminus \Gamma_1)\sigma$ and $\nabla \vdash Y\vartheta_2^n\sigma \approx (c\,a) \cdot t$ and $\nabla \vdash Y\vartheta_2^n\varphi \approx (c\,b) \cdot s$.

Since $\nabla \vdash c\#a.t$ and $\nabla \vdash c\#b.s$ it follows by application of rules in Figure 1 that $\nabla \vdash c.Y\vartheta_2^n\sigma \approx a.t$ and $\nabla \vdash c.Y\vartheta_2^n\varphi \approx b.s$. To conclude, from $\vartheta_1 = \{X/c.Y\}$ and $\Gamma_n \setminus \Gamma_0 = \Gamma_n \setminus \Gamma_1$, it follows that $\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1^n\rangle \preceq \langle \nabla, t_0\rangle$ and $\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1^n\rangle \preceq \langle \nabla, s_0\rangle$.

- The rule applied in the first step is (`Mer`):

  Then there exist $X : t_1 \triangleq s_1 \in S_0$ and $Y : t_2 \triangleq s_2 \in S_0$, $S_1 = S_0 \cup \{X : t_1 \triangleq s_1\}$, $\Gamma_1 = \Gamma_0\{Y/\pi \cdot X\}$ and $\vartheta_1 = \{Y/\pi \cdot X\}$ where $\nabla \vdash \pi \cdot t_1 \approx t_2$ and $\nabla \vdash \pi \cdot s_1 \approx s_2$. The first step is as follows:

  $$\emptyset; S'_0 \cup \{X : t_1 \triangleq s_1, Y : t_2 \triangleq s_2\}; \Gamma_0; \vartheta_0 \Longrightarrow \emptyset; S_1; \Gamma_1; \vartheta_0\vartheta_1$$

  By applying the IH in D, for every triple $Z_1 : t'_1 \triangleq s'_1 \in S_1$ the following hold: (i) $\langle \Gamma_n \setminus \Gamma_1, Z_1\vartheta_2^n\rangle \preceq \langle \nabla, t'_1\rangle$, and (ii) $\langle \Gamma_n \setminus \Gamma_1, Z_1\vartheta_2^n\rangle \preceq \langle \nabla, s'_1\rangle$. In particular, for the triple $X : t_1 \triangleq s_1 \in S_1$, that is, both relations $\langle \Gamma_n \setminus \Gamma_1, X\vartheta_2^n\rangle \preceq \langle \nabla, t_1\rangle$ and $\langle \Gamma_n \setminus \Gamma_1, X\vartheta_2^n\rangle \preceq \langle \nabla, s_1\rangle$ hold. Thus, there exist σ and φ respecting $\Gamma_n \setminus \Gamma_1$ such that: $\nabla \vdash (\Gamma_n \setminus \Gamma_1)\sigma$ and $\nabla \vdash (\Gamma_n \setminus \Gamma_1)\varphi$. $\nabla \vdash X\vartheta_2^n\sigma \approx t_1$ and $\nabla \vdash X\vartheta_2^n\sigma \approx s_1$

  We want to prove that for every $Z_0 : t_0 \triangleq s_0 \in S_0$ the following hold: (i') $\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1^n\rangle \preceq \langle \nabla, t_0\rangle$ and (ii') $\langle \Gamma_n \setminus \Gamma_1, Z_0\vartheta_0^n\rangle \preceq \langle \nabla, s_0\rangle$. Note that it remains to prove for the triples $X : t_1 \triangleq s_1, Y : t_2 \triangleq s_2$, all the other cases follow by the IH. For both (i') and (ii'), we verify that there exist σ and φ that respects $\Gamma_n \setminus \Gamma_0$ s.t. $\nabla \vdash (\Gamma_n \setminus \Gamma_0)\sigma$ and $\nabla \vdash (\Gamma_n \setminus \Gamma_0)\varphi$ hold.

  By the construction of the derivation D, the triple $X : t_1 \triangleq s_1$ is either retained in S_n, or is removed from there because there exist a triple $Z : t_n \triangleq s_n \in S_n$ and a permutation ρ such that $\nabla \vdash \rho \cdot t_n \approx t_1$, $\nabla \vdash \rho \cdot t_n \approx s_1$, and $X\vartheta_2^n = \rho \cdot Z$. Let's consider the second case, i.e., the following hold:

  a) $X\vartheta_1^n = X\vartheta_2^n = \rho \cdot Z$ and $Y\vartheta_1^n = (\pi \cdot X)\vartheta_2^n = \pi\rho \cdot Z$,

  b) $\Gamma_n \setminus \Gamma_0 = \{(\pi\rho)^{-1}a\#Z \mid a\#Y \in \Gamma_0\}$;

c) $\nabla \vdash \rho \cdot t_n \approx t_1$ and $\nabla \vdash \rho \cdot s_n \approx s_1$.

d) $\nabla \vdash \pi\rho \cdot t_n \approx t_2$ and $\nabla \vdash \pi\rho \cdot s_n \approx s_2$,

Now, take σ such that $Z\sigma = t_n$ and $b\#U \in \Gamma_n \setminus \Gamma_0$ arbitrary. By the IH, we have $\nabla \vdash \{b\#U\}\sigma$ for all $b\#U \in \Gamma_n \setminus \Gamma_1$. Note that $Z\sigma = t_n$ does not restrict generality, because if $U = Z$, then by Lemma 1 we have that $b\#U \in \Gamma_n \setminus \Gamma_1$ implies $\nabla \vdash b\#t_n$. Thus, $\nabla \vdash \{b\#U\}\sigma$ holds.

Now assume $b\#U \in (\Gamma_n \setminus \Gamma_0) \setminus (\Gamma_n \setminus \Gamma_1)$. Then $b\#U \in \Gamma_n \cap (\Gamma_1 \setminus \Gamma_0)$. That means, $b\#U = \pi^{-1} \cdot a\#X$, where $a\#Y \in \Gamma_0$. Moreover, the triple $X : t_1 \triangleq s_1$ has been retained in S_n. From the latter we have, in fact, $Z = X$, $t_n = t_1$, and $s_n = s_1$. On the one hand, we have $\{b\#U\}\sigma = \pi^{-1} \cdot a\#X\sigma = a\#\pi\cdot t_1$, and $a\#Y \in \Gamma_0$. On the other hand, from the assumption on $P_0; S_0; \Gamma_0; \vartheta_0$ we know that for all $a\#Y \in \Gamma_0$ we have $\nabla \vdash a\#t_2$, from which by $\nabla \vdash \pi \cdot t_1 \approx t_2$ we get $\nabla \vdash a\#\pi \cdot t_1$, which implies $\nabla \vdash \{b\#U\}\sigma$. Hence, $\nabla \vdash (\Gamma_n \setminus \Gamma_0)\sigma$. The proof that $\nabla \vdash (\Gamma_n \setminus \Gamma_0)\varphi$ holds is similar.

It remains to prove $\nabla \vdash Z_0\vartheta_1^n\sigma \approx t_0$ and $\nabla \vdash Z_0\vartheta_1^n\varphi \approx s_0$:

- Assume $Z_0 = X$, $t_0 = t_1$, $s_0 = s_1$.
  Then $\nabla \vdash Z_0\vartheta_2^n\sigma \approx t_0$, since from a) it follows that $Z_0\vartheta_2^n\sigma = X\vartheta_2^n\sigma = \rho \cdot Z\sigma = \rho \cdot t_n$ and from c) $\nabla \vdash \rho \cdot t_n \approx t_1$. Since $Z_0\vartheta_2^n = Z_0\vartheta_1^n$, we get $\nabla \vdash Z_0\vartheta_1^n\sigma \approx t_1$. The case $\nabla \vdash Z_0\vartheta_1^n\varphi \approx s_1$ is similar.
- Assume $Z_0 = Y$, $t_0 = t_2$, $s_0 = s_2$.
  From a) it follows that $Z_0\vartheta_1^n\sigma = Y\vartheta_1^n\sigma = \pi\rho \cdot Z\sigma = \pi\rho \cdot t_n$. From d) it follows that $\nabla \vdash \pi\rho \cdot t_n \approx t_2$. Hence, $\nabla \vdash Z_0\vartheta_1^n\sigma \approx t_2$ is proved. The case $\nabla \vdash Z_0\vartheta_1^n\varphi \approx s_2$ can be proved similarly.

□

Completeness states that for any given A-based generalization of two A-based input terms-in-context, there is a derivation in $\mathtt{NAU}_\emptyset$ that computes a (not strictly) less general A-based generalization.

Theorem 4 (Completeness of $\mathtt{NAU}_\emptyset$)**.** *Given terms t and s and freshness contexts ∇ and Γ, all based on a finite set of atoms A. If $\langle \Gamma, r\rangle$ is an A-based generalization of $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$, then there exists a derivation*

$$\{X : t \triangleq s\};\ \emptyset;\ \emptyset;\ \varepsilon \overset{+}{\Longrightarrow}_{\mathfrak{R}} \emptyset;\ S; \Gamma';\ \sigma$$

obtained by an execution of $\mathtt{NAU}_\emptyset$, such that $\langle \Gamma, r\rangle \preceq \langle \Gamma', X\sigma\rangle$.

Uniqueness, together with the former properties of $\texttt{NAU}_\emptyset$, confirms that the anti-unification problem in the empty theory is unitary.

Theorem 5 ($\texttt{NAU}_\emptyset$: Uniqueness Modulo $\approx$)**.** *Let t and s be terms and ∇ be a freshness context that are based on the same finite set of atoms. Let*

$$\{X : t \triangleq s\};\ \emptyset;\ \emptyset;\ \varepsilon \Longrightarrow^+ \emptyset;\ S_1; \Gamma_1;\ \sigma_1$$

and $\{X : t \triangleq s\}$; $\emptyset$; $\emptyset$; $\varepsilon \Longrightarrow^+ \emptyset$; S_2; Γ_2; σ_2 be two maximal derivations in $\texttt{NAU}_\emptyset$. Then $\langle \Gamma_1, X\sigma_1 \rangle \simeq \langle \Gamma_2, X\sigma_2 \rangle$.

The complexity theorem establishes that $\texttt{NAU}_\emptyset$ is tractable, i.e., its running time is polynomial in the input size.

Theorem 6 (Complexity of $\texttt{NAU}_\emptyset$)**.** *The nominal anti-unification algorithm $\texttt{NAU}_\emptyset$ has $\mathcal{O}(n^5)$ time complexity and $\mathcal{O}(n^4)$ space complexity, where n is the input size.*

4. Nominal Anti-Unification Modulo *A*-, *C*- and *AC*

In this section, we present the nominal equational anti-unification algorithm $\texttt{NAU}_E$ which extends the approach proposed in [7] with specialized rules for handling the equational theories A, C, and AC. Additionally, the merge rule (`Mer`) is specialised to the relation $\approx_E$. The decomposition rules closely follow those introduced in [14] for first-order anti-unification and in [4] for higher-order pattern anti-unification.

We only describe one representative rule in detail, as the behavior of the others is analogous. Throughout the application of the equational decomposition rules, the store S and the freshness context Γ remain unchanged.

(`DecA`) **Associative Decomposition**

$$\begin{aligned}&\{X : f^A(\overline{t})_n \triangleq f^A(\overline{s})_m\} \uplus P; S; \Gamma; \sigma \\ &\qquad \Longrightarrow \left\{\begin{array}{l} Y_1 : f^A(\overline{t})_k \triangleq f^A(\overline{s})_l, \\ Y_2 : f^A(\overline{t})_{k+1..n} \triangleq f^A(\overline{s})_{l+1..m} \end{array}\right\} \cup P; S; \Gamma; \sigma\{X/f^A(Y_1, Y_2)\},\end{aligned}$$

where Y_1 and Y_2 are new variables, $1 \leq k < n$ and $1 \leq l < m$.

Description: The rule (`DecA`) applies when both terms (already in flattened form) in the AUT_E are headed with the same associative function symbol

f^A. Note that it is possible that the terms in each side have different number of arguments, and decomposition should account for all possible forms of association of the arguments of f^A, in both sides.

(`DecC`): **Commutative Decomposition**

$$\{X : f^C(t_1, t_2) \triangleq f^C(s_1, s_2)\} \uplus P; S; \Gamma; \sigma \\ \Longrightarrow \left\{\begin{matrix} Y_1 : t_1 \triangleq s_i, \\ Y_2 : t_2 \triangleq s_{3-i} \end{matrix}\right\} \cup P; S; \Gamma; \sigma\{X/f^C(Y_1, Y_2)\},$$

where Y_1 and Y_2 are new variables.

(`DecAC`) **Associative-Commutative Decomposition**

$$\{X : f^{AC}(\overline{t})_n \triangleq f^{AC}(\overline{s})_m\} \uplus P; S; \Gamma; \sigma \\ \Longrightarrow \left\{\begin{matrix} Y_1 : f^{AC}(\overline{t})_{i_1..i_k} \triangleq f^{AC}(\overline{s})_{j_1..j_l}, \\ Y_2 : f^{AC}(\overline{t})_{i_{k+1}..i_n} \triangleq f^{AC}(\overline{s})_{j_{l+1}..j_m} \end{matrix}\right\} \cup P; S; \Gamma; \sigma\{X/f^{AC}(Y_1, Y_2)\},$$

where Y_1 and Y_2 are new variables, $\{i_1 \ldots, i_n\} \equiv \{1, \ldots, n\}, \{j_1 \ldots, j_m\} \equiv \{1, \ldots, m\}, 1 \leq k < n, 1 \leq l < m$.

The rule (`DecAC`) works on flattened terms. It needs to consider all the possible groupings of the arguments and all their permutations.

4.1. Generalization computation with $\mathtt{NAU}_E$

To compute a set of A-based E-generalizations of two given A-based terms-in-context $\langle \nabla, t\rangle$ and $\langle \nabla, s\rangle$, we proceed in the same manner as in the case of $\mathtt{NAU}_\emptyset$. That is, (i) the initial state is created (see Subsection 3.2), (ii) the rules are applied exhaustively, and (iii) the final state contains the computed generalization, as well as all the differences of the input terms-in-context. In contrast to $\mathtt{NAU}_\emptyset$, decomposition of $\mathtt{NAU}_E$ involves branching. That is, given an AUT_E where decomposition is applicable, the corresponding rule (`Dec`), (`DecA`), (`DecC`) or (`DecAC`) has to be applied in all possible ways, and all the results are being collected. Note that in the case of (`Dec`) there is only one way. The following examples illustrate that process. We start with the computation of the C-generalization from example in the Introduction:

Example 7 (Generalize pairs in 2)**.** *Consider the global parameters* $\nabla = \{a\#X, b\#X\}$ *and* $A = \{a, b, c\}$*. Let* $s = \mathtt{implies}(a.(\mathtt{P}(a) \wedge X), X)$ *and* $t =$

$\texttt{implies}(b.(X \wedge \texttt{P}(b)), X)$ *be the terms to be generalized modulo commutativity of* $\wedge$. *The initial state is* $\{X' : s \triangleq t\}; \emptyset; \emptyset; \varepsilon$. *We apply the decomposition rule and obtain:*

$$\begin{aligned}&\{X' : s \triangleq t\}; \emptyset; \emptyset; \varepsilon\\ &\Longrightarrow_{(\texttt{Dec})} \{X_1 : a.(\texttt{P}(a) \wedge X) \triangleq b.(X \wedge \texttt{P}(b)), X_2 : X \triangleq X\}; \emptyset; \emptyset; \sigma\end{aligned}$$

where X_1 *and* X_2 *are fresh variables,* $\sigma = \{X'/\texttt{implies}(X_1, X_2)\}$. *We note that* `implies` *is a binary symbol and* `P` *is a unary symbol in our signature and have no semantic meaning. Next, we apply the abstraction rule and obtain:*

$$\Longrightarrow_{(\texttt{Abs})} \{Y : (\texttt{P}(a) \wedge X) \triangleq ((b\,a) \cdot X \wedge \texttt{P}(a)), X_2 : X \triangleq X\}; \emptyset; \emptyset; \sigma_1$$

where $\sigma_1 = \sigma\{X_{11}/a.Y\}$ *and* Y *is a fresh variable. Note that in* ∇ *we only have the information that* $a, b\#X$, *we could have chosen the name* b *instead of* a *here. Also, if we had* $c\#X$ *in* ∇ *we could have used atom* c *for the abstraction.*

Now, we apply the commutative decomposition rule which will give us two branches (B1) and (B2) below:

$$\Longrightarrow_{(\texttt{DecC})} \{Y_1 : \texttt{P}(a) \triangleq (b\,a) \cdot X, Y_2 : X \triangleq \texttt{P}(a), X_2 : X \triangleq X\}; \emptyset; \emptyset; \sigma_2 \quad \text{(B1)}$$

$$\{Y_1 : \texttt{P}(a) \triangleq \texttt{P}(a), Y_2 : X \triangleq (b\,a) \cdot X, X_2 : X \triangleq X\}; \emptyset; \emptyset; \sigma_2 \quad \text{(B2)}$$

where $\sigma_2 = \sigma_1\{Y/Y_1 \wedge Y_2\}$ *and* Y_1 *and* Y_2 *are fresh variables. We proceed with the full computation of branch (B2). After applying the decomposition rule twice followed by two application of the solve rule we obtain:*

$$\begin{aligned}&\Longrightarrow^2_{(\texttt{Dec})} \{Y_2 : X \triangleq (b\,a) \cdot X, X_2 : X \triangleq X\}; \emptyset; \emptyset; \sigma_3\\ &\Longrightarrow^2_{(\texttt{Sol})} \emptyset; \{Y_2 : X \triangleq (b\,a) \cdot X, X_2 : X \triangleq X\}; \{a\#X_2, b\#X_2, a\#Y_2, b\#Y_2\}; \sigma_3\end{aligned}$$

where $\sigma_3 = \sigma_2\{Y_1/\texttt{P}(a)\}$. *Finally, we apply the merge rule and obtain:*

$$\Longrightarrow_{(\texttt{Mer})} \emptyset; \{Y_2 : X \triangleq (b\,a) \cdot X\}; \{a\#Y_2, b\#Y_2\}; \sigma_4$$

where $\sigma_4 = \sigma_3\{X_2/id \cdot Y_2\}$ *and no more rules can be applied. Note that* $\nabla \vdash id \cdot X \approx_C X$ *and* $\nabla \vdash id \cdot ((a\,b) \cdot X) \approx_C X$. *Thus, branch (B2) computes the A-based C-generalization:*

$$\langle \{a\#Y_2, b\#Y_2\}, \texttt{implies}(a.(\texttt{P}(a) \wedge Y_2), Y_2) \rangle \quad (3)$$

The branch (B1) computes the C-generalization

$$\langle\{b\#Y_1, b\#Y_2, a\#X_2, b\#X_2\}, \texttt{implies}(a.(Y_1 \wedge Y_2), X_2)\rangle$$

which is more general than the C-generalization in 3.

Example 8. *Let* $A = \{a, b, c, d, a'\}$ *and* $\nabla = \{a'\#Z, a'\#Y\}$ *be the global parameters. Consider the terms* $t = a.f^A(a, b, Y)$ *and* $s = b.f^A(Z, b, c, d)$ *in flattened form. The initial state is* $\{X : t \triangleq s\}; \emptyset; \emptyset; \varepsilon$*. First, we apply the abstraction rule, obtaining*

$$\begin{aligned}&\{X : a.f^A(a, b, Y) \triangleq b.f^A(Z, b, c, d)\}; \emptyset; \emptyset; \varepsilon\\ &\Longrightarrow_{(\texttt{Abs})} \{X' : f^A(a', b, (a\ a') \cdot Y) \triangleq f^A((b\ a') \cdot Z, a', c, d)\}; \emptyset; \emptyset; \{X/a'.X'\}.\end{aligned}$$

Then the associative decomposition rule is applied, resulting in 6 new states.

$$\Longrightarrow_{(\texttt{DecA})} P_i; \emptyset; \emptyset; \sigma\{X/a'.f^A(Y_1, Y_2)\},$$

where Y_1, Y_2 *are fresh variables and the problem sets of the new states are:*

(P_1*)* $k = 1, l = 1$*:* $\{Y_1 : a' \triangleq (b\ a') \cdot Z, Y_2 : f^A(b, (a\ a') \cdot Y) \triangleq f^A(a', c, d)\}$

(P_2*)* $k = 1, l = 2$*:* $\{Y_1 : a' \triangleq f^A((b\ a') \cdot Z, a'), Y_2 : f^A(b, Y) \triangleq f^A(c, d)\}$

(P_3*)* $k = 1, l = 3$*:* $\{Y_1 : a' \triangleq f^A((b\ a') \cdot Z, a', Y), Y_2 : f^A(b, Y) \triangleq d\}$

(P_4*)* $k = 2, l = 1$*:* $\{Y_1 : f^A(a', b) \triangleq (b\ a') \cdot Z, Y_2 : (a\ a') \cdot Y \triangleq f^A(a', c, d)\}$

(P_5*)* $k = 2, l = 2$*:* $\{Y_1 : f^A(a', b) \triangleq f^A((b\ a') \cdot Z, a'), Y_2 : (a\ a') \cdot Y \triangleq f^A(c, d)\}$

(P_6*)* $k = 2, l = 3$*:* $\{Y_1 : f^A(a', b) \triangleq f^A((b\ a') \cdot Z, a', c), Y_2 : (a\ a') \cdot Y \triangleq d\}$

From this point, we can proceed by applying (`Sol`) *or* (`DecA`)*, leading to new problems along different branches. For instance, one E-generalization obtained from (*P_1*) is* $\langle\{a'\#Z, a'\#Y, b\#Y_1, a\#Y_{21}, a\#Y_{22}\}, f^A(Y_1, f^A(Y_{21}, Y_{22}))\rangle$*, where* Y_{21} *and* Y_{22} *are fresh variables. The E-generalization obtained from (*P_4*) is* $\langle\{a'\#Z, a'\#Y, b\#Y_1, a\#Y_2\}, f^A(Y_1, Y_2)\rangle$*. The first one is less general than the second.*

Example 9. *Consider the global parameters* $A = \{a, b\}$ *and* $\nabla = \emptyset$*. Let* $t = f^{AC}((a\,b) \cdot X', g(a, a, b), g(b, b, a))$ *and* $s = f^{AC}(h^{AC}(b, a, b), h^{AC}(a, a, b), X')$

be the terms to be generalized. The initial state is $\{X : t \triangleq s\}; \emptyset; \emptyset; \varepsilon$. *We apply the AC-decomposition rule, and one of the branches is*

$$\begin{aligned}
&\{X : t \triangleq s\}; \emptyset; \emptyset; \varepsilon \\
&\Longrightarrow_{(\mathtt{DecAC})} \left\{\begin{array}{l} Y_1 : (a\,b) \cdot X' \triangleq X', \\ Y_2 : f^{AC}(g(a,a,b), g(b,b,a)) \triangleq f^{AC}(h^{AC}(b,a,b), h^{AC}(a,a,b)) \end{array}\right\}; \\
&\qquad \emptyset; \emptyset; \{X/f^{AC}(Y_1, Y_2)\} \\
&\Longrightarrow_{(\mathtt{DecAC})} \left\{\begin{array}{l} Y_1 : (a\,b) \cdot X' \triangleq X', \\ Z_1 : g(a,a,b) \triangleq h^{AC}(b,a,b), \\ Z_2 : g(b,b,a) \triangleq h^{AC}(a,a,b) \end{array}\right\}; \emptyset; \emptyset; \{X/f^{AC}(Y_1, Z_1, Z_2)\}.
\end{aligned}$$

Next, the solve rule is applied three times, obtaining

$$\begin{aligned}
&\Longrightarrow^3_{\mathsf{Sol}} \emptyset; \left\{\begin{array}{l} Y_1 : (a\,b) \cdot X' \triangleq X', \\ Z_1 : g(a,a,b) \triangleq h^{AC}(b,a,b), \\ Z_2 : g(b,b,a) \triangleq h^{AC}(a,a,b) \end{array}\right\}; \emptyset;\ \{X/f^{AC}(Y_1, Z_1, Z_2)\} \\
&\Longrightarrow_{\mathsf{Mer}} \emptyset; \left\{\begin{array}{l} Y_1 : (a\,b) \cdot X' \triangleq X', \\ Z_1 : g(a,a,b) \triangleq h^{AC}(b,a,b) \end{array}\right\}; \emptyset; \{X/f^{AC}(Y_1, Z_1, (a\,b) \cdot Z_1)\}.
\end{aligned}$$

Note that, in the merge step, the permutation $\pi = (a\ b)$ *satisfies* $\emptyset \vdash (a\ b) \cdot g(a, a, b) \approx_E g(b,b,a)$ *and* $\emptyset \vdash (a\ b) \cdot h^{AC}(b,a,b) \approx_E h^{AC}(a,a,b)$. *The computation of* π *is shown in Example 10. The computed* E*-generalization is* $\langle \emptyset, f^{AC}(Y_1, Z_1, (a\,b) \cdot Z_1) \rangle$ *and the input terms* t, s *can be retrieved by applying, respectively, the substitutions* σ_1 *and* σ_2 *to* $f^{AC}(Y_1, Z_1, (a\,b) \cdot Z_1)$, *where* $\sigma_1 = \{Y_1/(a\,b) \cdot X', Z_1/g(a,a,b)\}$ *and* $\sigma_2 = \{Y_1/X', Z_1/h^{AC}(b,a,b)\}$. *Note that* σ_1 *and* σ_2 *are constructed from information in the store.*

4.2. Properties of the $\mathtt{NAU}_E$*-algorithm*

In this section, we establish the key properties of the $\mathtt{NAU}_E$ algorithm. Termination of $\mathtt{NAU}_E$ follows from the termination of $\mathtt{NAU}_\emptyset$ (Theorem 2) as the new decomposition rules, although branching, always generate strictly simpler problems. Soundness and completeness of $\mathtt{NAU}_E$ are inherited from their counterparts in $\mathtt{NAU}_\emptyset$ (Theorem 3 and Theorem 4) and the fact that the equational decomposition rules are known to be sound and complete [15].

Theorem 7. $\mathtt{NAU}_E$ *is sound.*

Proof. The result follows by extending the proof of Theorem 3 with the equational decomposition rules, and replacing the relations $\approx, \preceq$ with $\approx_E, \preceq_E$ where appropriate. We only present the inductive step for rule (`DecC`) as the proofs for (`DecA`) and (`DecAC`) follow the same reasoning and are exhaustive. The rule applied in the first step is (`DecC`):

Then $Z_0 = X$, $t_0 = f^C(t_1, t_2)$, $s_0 = f^C(s_1, s_2)$, and the problem is $P_0 = \{X : f^C(t_1, t_2) \triangleq f^C(s_1, s_2)\} \uplus P'_0$. Also, $\Gamma_1 = \Gamma_0$, $S_0 = S_1$ and $\vartheta_1 = \{X/f^C(Y_1, Y_2)\}$. Thus, the first step branches as follows:

$$\{X : f^C(t_1, t_2) \triangleq f^C(s_1, s_2)\} \uplus P'_0; S_0; \Gamma_0; \vartheta_0 \Longrightarrow P_1;\ S_1; \Gamma_1; \vartheta_0\vartheta_1$$

where P_1 contains either one of the AUT_Es: (i) $\{Y_1 : t_1 \triangleq s_1, Y_2 : t_2 \triangleq s_2\}$; or (ii) $\{Y_1 : t_1 \triangleq s_2, Y_2 : t_2 \triangleq s_1\}$. We will prove the case for (ii) as case (i) is analogous to the case above. Thus, the rest of the proof considers the branch in which the state contains the problem $P_1 = \{Y_1 : t_1 \triangleq s_2, Y_2 : t_2 \triangleq s_1\} \cup P'_1$. By applying the IH in the derivation

$$\mathsf{D}_1 :\ \{Y_1 : t_1 \triangleq s_2, Y_2 : t_2 \triangleq s_1\} \cup P'_1;\ S_1; \Gamma_1; \vartheta_0\vartheta_1 \Longrightarrow^{n-1}_{\mathfrak{R}} \emptyset;\ S_n; \Gamma_n; \vartheta_0^n$$

it follows that the following hold for $i = 1, 2$: $\langle \Gamma_n \setminus \Gamma_1, Y_i\vartheta_2^n \rangle \preceq_E \langle \nabla, t_i \rangle$, and $\langle \Gamma_n \setminus \Gamma_1, Y_i\vartheta_2^n \rangle \preceq_E \langle \nabla, s_{3-i} \rangle$ hold. Hence, by definition of $\preceq_E$, there exist σ and φ (given by the store S_n) that respect $\Gamma_n \setminus \Gamma_1$ and satisfy the following: $(\Gamma_n \setminus \Gamma_1)\sigma \subseteq \nabla$ and $(\Gamma_n \setminus \Gamma_1)\varphi \subseteq \nabla$, and $\nabla \vdash Y_i\vartheta_2^n\sigma \approx_E t_i$ and $\nabla \vdash Y_i\vartheta_2^n\varphi \approx_E s_{3-i}$ for $i = 1, 2$. Finally, since $\vartheta_1 = \{X/f^C(Y_1, Y_2)\}$ and $\Gamma_n \setminus \Gamma_0 = \Gamma_n \setminus \Gamma_1$, we obtain $\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1^n \rangle \preceq_E \langle \nabla, t_0 \rangle$ and $\langle \Gamma_n \setminus \Gamma_0, Z_0\vartheta_1^n \rangle \preceq_E \langle \nabla, s_0 \rangle$.

□

There can be different derivations in $\mathtt{NAU}_E$ that start from the same AUT_E, leading to different generalizations. It is worth noting that achieving minimality was not the goal of the algorithm discussed above. Consequently, the resulting complete set of generalizations is not guaranteed to be minimal.

Theorem 8. $\mathtt{NAU}_E$ *is complete.*

Proof. The result follows by extending the proof of Theorem 4 with the equational decomposition rules, and replacing the relations $\approx, \preceq$ with $\approx_E, \preceq_E$ where appropriate. Completeness of the A-, C- and AC-decomposition rules follow from the fact that these rules expand all possibilities of branching [15].

□

Theorem 9. $\mathtt{NAU}_E$ *is finitary.*

Proof. We build upon the minimal complete set of generalizations computed by $\mathtt{NAU}_\emptyset$ that is unitary. The A-, C- and AC-decomposition rules introduce a finite number of branches. Thus, from Theorem 8, it follows that our algorithm computes a finite and complete set of E-generalizations, which includes the minimal set. □

5. EQVM: The Equivariance Algorithm

In this section we present the algorithm EQVM that is used to compute the permutation π required by the merging rule (Mer) of $\mathtt{NAU}_E$. To determine if such a permutation exists, we must solve the following problem.

Given: two anti-unification triples $X_1 : t_1 \triangleq s_1$ and $X_2 : t_2 \triangleq s_2$, a freshness context ∇ and a finite set of atoms A.
Return: an A-based permutation π such that $\nabla \vdash \pi \cdot t_1 \approx_E t_2$ and $\nabla \vdash \pi \cdot s_1 \approx_E s_2$, or $\perp$ if no such permutation exists.

Intuitively, we take the triples $X_1 : t_1 \triangleq s_1$ and $X_2 : t_2 \triangleq s_2$ from $\mathtt{NAU}_E$ and construct the *equivariance problem* $\{t_1 \sqsubseteq t_2, s_1 \sqsubseteq s_2\}$, ∇, A that serves as input of EQVM. In fact, the equivariance problem is not restricted to input sets of cardinality two. Finite sets of pairs of the form $t \sqsubseteq t'$ may be considered to search for a permutation π that equates them modulo E, considering ∇ and A. To solve that problem we extend the nominal equivariance algorithm that was introduced for the empty theory in [7] to take into account equational theories A, C and AC.

We present the rules for the EQVM algorithm in Figure 3. These rules operate on triples of the form $\mathcal{E}$; A; π, called states. $\mathcal{E}$ is the problem set, A is a finite set of allowed atoms, and π is the permutation which is returned in case of success. The freshness context ∇ is treated as a global parameter throughout the execution. Again, we assume that associative function applications are flattened. There is no separate decomposition rule for associative symbols, since, in that case, flattening of associative function applications and simply applying syntactic decomposition suffices.

Remark 3. *Note that the rules in Figure 3 that have to do with decomposition modulo A or AC have the same number of arguments in both sides. This is because the application of a permutation π in a flattened term, say*

Phase 1. Simplifying the equivariance problem.
$(\texttt{Dec}-\texttt{E})$ / $(\texttt{DecA}-\texttt{E})$: $\{f(t_1,\ldots,t_m) \sqsubseteq f(s_1,\ldots,s_m)\} \uplus \mathcal{E};\ A;\ Id \Longrightarrow \bigcup_{k=1}^{m}\{t_k \sqsubseteq s_k\} \cup \mathcal{E};\ A;\ Id.$
$(\texttt{DecC}-\texttt{E})$: $\{f^C(t_1,t_2) \sqsubseteq f^C(s_1,s_2)\} \uplus \mathcal{E};\ A;\ Id \Longrightarrow \{t_1 \sqsubseteq s_i,\ t_2 \sqsubseteq s_{3-i}\} \cup \mathcal{E};\ A;\ Id.$
$(\texttt{DecAC}-\texttt{E})$: $\{f^{AC}(\bar{t})_n \sqsubseteq f^{AC}(\bar{s})_n\} \uplus \mathcal{E};\ A;\ Id \Longrightarrow$ $\{t_1 \sqsubseteq s_j, f^{AC}(t_2,\ldots,t_n) \sqsubseteq f^{AC}(s_1,\ldots,s_{j-1},s_{j+1},\ldots,s_n)\} \cup \mathcal{E};\ A;\ Id.$
$(\texttt{Abs}-\texttt{E})$: $\{a.t \sqsubseteq b.s\} \uplus \mathcal{E};\ A;\ Id \Longrightarrow \{(c\ a)\cdot t \sqsubseteq (c\ b)\cdot s\} \cup \mathcal{E};\ A;\ Id.$
$(\texttt{Sus}-\texttt{E})$: $\{\pi_1 \cdot X \sqsubseteq \pi_2 \cdot X\} \uplus \mathcal{E};\ A;\ Id \Longrightarrow$ $\{\pi_1 \cdot a \sqsubseteq \pi_2 \cdot a \mid a \in A \wedge a\#X \notin \nabla\} \cup \mathcal{E};\ A;\ Id.$
Phase 2. Computing the permutation.
$(\texttt{Rem}-\texttt{E})$: $\{a \sqsubseteq b\} \uplus \mathcal{E}; A; \pi \Longrightarrow \mathcal{E}; A \setminus \{b\};\ \pi,$ if $\pi \cdot a = b.$
$(\texttt{Sol}-\texttt{E})$: $\{a \sqsubseteq b\} \uplus \mathcal{E}; A; \pi \Longrightarrow \mathcal{E}; A \setminus \{b\};\ (\pi \cdot a\ b)\pi,$ if $\pi \cdot a \neq b$ and $\{\pi \cdot a,\ b\} \subseteq A.$

Figure 3: Equivariance Rules for `EQVM`. Here, a and b are not necessarily different atoms, $m \geq 0$, $n \geq 2$, $i \in \{1,2\}$, $1 \leq j \leq n$ and c is a fresh auxiliary atom of the corresponding sort that does not occur in A

$f^A(s_1,\ldots,s_n)$, cannot alter the number of arguments of f^A. The permutation action is as follows:

$$\pi \cdot f^A(s_1,\ldots,s_n) = f^A(\pi \cdot s_1,\ldots,\pi \cdot s_n)$$

Since the arguments of f^A cannot be permuted, the equivariant associative decomposition rule behaves syntactically. With AC-function symbols we also have the same number of arguments in both sides, but those can be permuted.

5.1. Permutation computation with `EQVM`

The `EQVM` algorithm works in two phases. Phase 1 decomposes and simplifies the given equivariance problem. Phase 2 computes the permutation.

Given an equivariance problem $\{t_1 \sqsubseteq t'_1, \ldots, t_n \sqsubseteq t'_n\}$, ∇, A. To run the algorithm, we create the initial state $\{t_1 \sqsubseteq t'_1, \ldots, t_n \sqsubseteq t'_n\}$; A; Id and apply the rules of Phase 1 exhaustively. Due to commutative decomposition, it involves branching. When no more rule of Phase 1 can be applied to a state then Phase 2 starts and we apply those rules exhaustively too. To a final state no more rule can be applied. The final state is a success state if the problem set is empty. I.e., a branch succeeds if it reaches a state of the form $\emptyset$; A'; π. In that case π is the computed permutation. On the other hand, if the problem set of a finial state is not empty it fails with $\perp$.

The entire procedure succeeds if one of the branches succeeds and, in that case, it returns a permutation computed by a successful branch. Note that there might be more than one successful branches.

Example 10. *To obtain the permutation $\pi = (a\ b)$ in the last step of Example 9, we proceeded as follows:*

$$\{g(a,a,b) \sqsubseteq g(b,b,a),\ h^{AC}(b,a,b) \sqsubseteq h^{AC}(a,a,b)\}; \emptyset; \{a,b\}; Id \Longrightarrow_{\mathsf{Dec-E}}$$
$$\{a \sqsubseteq b,\ b \sqsubseteq a,\ h^{AC}(b,a,b) \sqsubseteq h^{AC}(a,a,b)\}; \emptyset; \{a,b\}; Id \Longrightarrow^{i=1}_{\mathsf{DecAC-E}}$$
$$\{a \sqsubseteq b,\ b \sqsubseteq a,\ h^{AC}(a,b) \sqsubseteq h^{AC}(a,b)\}; \emptyset; \{a,b\}; Id \Longrightarrow^{i=2}_{\mathsf{DecAC-E}}$$
$$\{a \sqsubseteq b,\ b \sqsubseteq a\}; \emptyset; \{a,b\}; Id \Longrightarrow^{\mathsf{Sol-E}}_{\mathsf{Rem-E}} \qquad \emptyset; \emptyset; \emptyset; (a\,b)$$

The example illustrates a successful branch and the computed permutation is $(a\,b)$. Note that, in the third step, $h^{AC}(t)$ (h^{AC} applied to a single argument) stands for t.

5.2. Correctness of `EQVM`

Given two terms t, s, a freshness context ∇ and a finite set of atoms A. We prove that the algorithm `EQVM` terminates, computes only A-based permutations and it holds that

$$\nabla \vdash \mu \cdot s \approx_E t \quad \Longleftrightarrow \quad \pi \in \texttt{EQVM}(s \sqsubseteq t, \nabla),$$

where $\pi(a) = \mu(a)$, for all $a \in \mathrm{FA}(s)$. For convenience reasons, the correctness proof of Phase 2 is extracted in Lemma 2. Afterwards both directions are being considered separately in Theorem 11 and Theorem 12.

Theorem 10. *The algorithm* `EQVM` *terminates on any input.*

Proof. We consider a complexity measure on the set of equations E. It consists of three components. The first one is the sum of arguments under all the function applications that appear in E. The second component is the sum of all suspensions and abstractions that appear in E, and the third one is simply the number of equations in E. Every rule in Figure 3 strictly decreases the complexity measure. Since decomposition only generates finitely many branches, `EQVM` terminates. □

Lemma 2 (Correctness of Phase 2)**.** *Given a set of atom equations* $E = \{l_1 \sqsubseteq r_1, \ldots, l_n \sqsubseteq r_n\}$ *where* l_i, r_i, $1 \leq i \leq n$ *denote arbitrary atoms from* $\mathcal{A}$*, and a set* $A \subseteq \mathcal{A}$*. Phase 2 generates a permutation* π *such that* $\mathsf{supp}(\pi) \subseteq A$ *and* $\pi \cdot l_i = r_i$, $1 \leq i \leq n$ *if and only if such a permutation exists.*

Proof. First, note that Phase 2 starts with $\pi = Id$. The only rule that actually changes the so-far computed permutation is $(\mathtt{Sol} - \mathtt{E})$. The condition $\{\pi \cdot a,\, b\} \subseteq A$ of $(\mathtt{Sol} - \mathtt{E})$ ensures that $\mathsf{supp}(\pi) \subseteq A$. Cases of the form $\{a \sqsubseteq b, c \sqsubseteq b\}$ or $\{a \sqsubseteq b, b \sqsubseteq b\}$ won't lead to valid permutations, since, a function that maps $\{a \mapsto b, c \mapsto b\}$ or $\{a \mapsto b, b \mapsto b\}$ can't be a bijection. Therefore, in the process of Phase 2, the "already considered range" has to be reduced. I.e., by performing $A \setminus \{b\}$ the rules of Phase 2 in Figure 3 remove the currently considered atom b from the range to be considered. Note that it also covers cases of the form $\{a \sqsubseteq b, a \sqsubseteq c\}$ and $\{a \sqsubseteq b, a \sqsubseteq a\}$, since, the so-far computed bijection is extended by $\{a \mapsto b\}$ and $A \setminus \{b\}$ together with the condition $\{\pi \cdot a,\, b\} \subseteq A$ won't allow $\{(a\,b) \cdot a \sqsubseteq c\} = \{b \sqsubseteq c\}$ or $\{(a\,b) \cdot a \sqsubseteq a\}$ to be satisfied. It follows that, there remains at least one pair of atoms in the set of equations E, if and only if no bijection exists that satisfies all the pairs of atoms. I.e., an exhaustive rule application of Phase 2 leads to $E = \emptyset$ if and only if all the equations are being satisfied. □

Theorem 11 (Soundness of `EQVM`)**.** *Suppose there is a derivation as*

$$\{s \sqsubseteq t\}; \nabla; A; Id \overset{*}{\Longrightarrow}_{\mathtt{EQVM}} \emptyset; \nabla; B; \pi$$

then π *is an* A*-based permutation such that* $\nabla \vdash \pi \cdot s \approx_E t$*.*

Proof. The decomposition rules of `EQVM` simply correspond to those of the α-equivalence rules from Figure 2. Bound atoms must appear at the same positions in the sub-terms, otherwise no permutation exists. For that reason, in $(\mathtt{Abs} - \mathtt{E})$ an atom outside of A can be taken for renaming. That atom won't appear in the permutation (Lemma 2). The rule $(\mathtt{Sus} - \mathtt{E})$ reduces the

problem of finding a permutation π such that $\nabla \vdash \pi\pi_1 \cdot X \approx_E \pi_2 \cdot X$ to simple atom mappings. By Lemma 2, fulfilling the atom mappings corresponds to $a\#X \in \nabla$ for all $a \in A \cap ds(\pi\pi_1, \pi_2)$. I.e., it corresponds to the α-equivalence rule `Var` of Figure 2. Together with Lemma 2 it follows that π is an A-based permutation such that $\nabla \vdash \pi \cdot s \approx_E t$. □

Theorem 12 (Completeness of `EQVM`)**.** *Let A be a finite set of atoms, s, t be terms and ∇ be a freshness context. If $\nabla \vdash \mu \cdot s \approx_E t$ holds for some A-based permutation μ, then there exists a derivation*

$$\{s \sqsubseteq t\}; \nabla; A; Id \stackrel{*}{\Longrightarrow}_{\texttt{EQVM}} \emptyset; \nabla; B; \pi$$

such that $\pi(a) = \mu(a)$, for all $a \in FA(s)$.

Proof. Assuming $\nabla \vdash \mu \cdot s \approx_E t$, we show that one of the rule in `EQVM` is applicable whenever an α-equivalence rule from Figure 2 can be applied. Again, the correspondence of our decomposition rules to those from Figure 2 is straightforward. The case $\nabla \vdash \mu \cdot a \approx_E b$ where $s = a$ and $t = b$, is covered by Lemma 2. It remains to analyze the following cases of α-equivalence rules from Figure 2:

$$\frac{ds(\mu\pi_1, \pi_2)\#X \in \nabla}{\nabla \vdash \mu\pi_1 \cdot X \approx_E \pi_2 \cdot X}(\texttt{Var}) \qquad \frac{\nabla \vdash \mu \cdot s \approx_E t \qquad \mu \cdot a = b}{\nabla \vdash \mu \cdot a.s \approx_E b.t}(\texttt{Aba})$$

$$\frac{\nabla \vdash \mu \cdot s \approx_E (\mu \cdot a\, b) \cdot t \qquad \mu \cdot a \neq b \text{ and } \nabla \vdash \mu \cdot a\#t}{\nabla \vdash \mu \cdot a.s \approx_E b.t}(\texttt{Abb})$$

Since s, t and μ are A-based, if the rule `Var` is applicable then μ is a permutation such that $ds(\mu\pi_1, \pi_2) \cap A\#X \in \nabla$ is satisfied. By applying $\texttt{Sus} - \texttt{E}$ of `EQVM` the atom-equations $\{\pi_1 \cdot a \sqsubseteq \pi_2 \cdot a \mid a \in A \wedge a\#X \notin \nabla\}$ are being created. From Lemma 2 follows that π results in a permutation such that $ds(\pi\pi_1, \pi_2) \cap A\#X \in \nabla$. In $\texttt{Abs} - \texttt{E}$ a fresh atom $c \notin A$ is being taken, i.e., $\Gamma \cup \nabla \vdash c\#s$ and $\Gamma \cup \nabla \vdash c\#t$, where $\Gamma = \{c\#X \mid X$ is a variable that appears in s or $t\}$. By Lemma 2, c won't appear in the computed permutation π. Therefore, the cases of the α-equivalence rules `Abb` and `Aba` are covered by the `EQVM` rule $\texttt{Abs} - \texttt{E}$. □

6. Implementation with Branching Heuristics

The anti-unification algorithm $\texttt{NAU}_E$ and its subalgorithm `EQVM` have been implemented in Java and are freely available. They can also be tried out online. The website of the prototype implementation is

`https://nau-eq.web.app/`.

It is a brute-force implementation that iterates all branches. I.e., given a finite set of atoms A, two A-based terms t, s, and an A-based freshness context ∇, it finds all generalizations from the A-based $\mathtt{mcsg}_E(\nabla, t, s)$. Since no minimization has been implemented, the iteration may include duplicates and other A-based generalizations from $\mathcal{G}_E(\nabla, t, s)$ that are not least general. However, heuristics to choose promising branches first have been implemented. Enabling the heuristics, allows to trade completeness for tractability.

To illustrate the behavior, consider the $\{a, b, c\}$-based terms $f^{AC}(f^{AC}(a, b), c)$ and $f^{AC}(b, f^{AC}(c, a))$ with an empty freshness context. The brute-force method generates 27 generalizations. Figure 4 shows the first few generalizations being iterated without using any heuristics. The eighth branch is actually the first one that emits a least general generalization. As usual, associative function applications are flattened.

1. $\langle\{b\#Z4\}, f^{AC}((c\ a)(a\ b) \cdot Z4, (a\ b)(a\ c) \cdot Z4, Z4)\rangle$
2. $\langle\{c\#Z3\}, f^{AC}((a\ b) \cdot Z3, Z3, c)\rangle$
3. $\langle\{b\#Z1, a\#Z2\}, f^{AC}(Z1, Z2)\rangle$
4. $\langle\{c\#Z1, a\#Z2\}, f^{AC}(Z1, Z2)\rangle$
5. $\langle\{b\#Z6\}, f^{AC}((a\ c) \cdot Z6, b, Z6)\rangle$
6. $\langle\{a\#Z6\}, f^{AC}((c\ a)(a\ b) \cdot Z6, (a\ b)(a\ c) \cdot Z6, Z6)\rangle$
7. $\langle\emptyset, f^{AC}(Z1, Z2)\rangle$
8. $\langle\emptyset, f^{AC}(a, b, c)\rangle$
9. $\langle\{a\#Z8\}, f^{AC}(a, (b\ c) \cdot Z8, Z8)\rangle$

   ...

Figure 4: Input terms $f^{AC}(f^{AC}(a, b), c)$ and $f^{AC}(b, f^{AC}(c, a))$ without heuristics.

In the decomposition step (`DecAC`) all the combinations of partitionings of the arguments into 2 non-empty subsets have to be considered. More precisely, given $f^{AC}(\overline{t})_n \triangleq f^{AC}(\overline{s})_m$, decomposition generates $(2^{n-1} - 1) \times$

$(2^{m-1} - 1) \times 2$ branches. To implement the heuristics that lead to iterating promising branches first (e.g., branch number 8 in Figure 4), we need to fix a deterministic strategy when applying the rules.

6.1. Implemented Strategy

The following strategy is used to apply the rules of $\mathtt{NAU}_E$:

1. As long as the set of unsolved anti-unification triples P is not empty, select one of them and apply the corresponding rule. Note that the application of the merge rule is delayed as long as possible. Decomposition involves branching and the branches are queued.

2. When P is empty, apply the merging rule (`Mer`) as often as possible. The subalgorithm `EQVM` is used to solve the equivariance problem. `EQVM` involves branching. Nevertheless, all the possible permutations lead to equi-general results. Therefore, we simply use the first one (none deterministically), if there is any.

3. If the queue of branches is not empty, select the next branch and repeat the entire procedure.

The generation of the branches during the decomposition process needs to be deterministic. For the sake of simplicity, we concentrate on the first branch. I.e., the one that is used to continue immediately. Consider the decomposition rules of $\mathtt{NAU}_E$ and the terms $f^E(\overline{t})_n \triangleq f^E(\overline{s})_m$. The first branch generated by the implementation is the one that decomposes into

$$\left\{\begin{array}{l} Y_1 : t_1 \triangleq s_1, \\ Y_2 : f^E(\overline{t})_{2\ldots n} \triangleq f^E(\overline{s})_{2\ldots m} \end{array}\right\}, \text{ if } E \in \{A, AC\},$$

$$\left\{\begin{array}{l} Y_1 : t_1 \triangleq s_1, \\ Y_2 : t_2 \triangleq s_2 \end{array}\right\}, \text{ if } E = C.$$

Example 11. *Given the flattened terms $f^{AC}(a, b, c) \triangleq f^{AC}(b, c, a)$. The first branch that will be considered is the one that decomposes into $a \triangleq b$ and $f^{AC}(b, c) \triangleq f^{AC}(c, a)$. Decomposition applies again, and the implemented strategy leads to $a \triangleq b$, $b \triangleq c$ and $c \triangleq a$ being the first choice to be considered. The resulting generalization can be seen in Figure 4 line 1.*

The idea of the branching heuristics is to rearrange the arguments of function applications that are commutative (C and AC). That is, *aligning* the two sequences of arguments. Together with the deterministic decomposition strategy, promising branches will be computed first.

6.2. Aligning Commutative Arguments

In order to rearrange commutative arguments, a *total order on flattened A-based terms* is being introduced. We use $\trianglelefteq$ to denote this order. More precisely, a lexicographical order where each component consists of a mayor and a minor part is being employed. The mayor part is a number that simply depends on the type of the term. The numbers used in our prototype implementation have been chosen as: atom = 10, application = 20, suspension = 30, abstraction = 40. The minor part is the head of the term. In the case of function applications and abstractions the order is defined inductively on the term structure as described below. In the other two cases we are already done. Note that all abstractions have the same head.

Given two abstractions, the relation $\trianglelefteq$ is defined as $a.t \trianglelefteq b.s$ if and only if $(a\,c) \cdot t \trianglelefteq (b\,c) \cdot s$, where c is a fresh auxiliary atom that does not occur in A.[†] Moreover, the fresh atom c has the highest ordinal number among all the atoms in t and s. Similarly, $a.t \trianglelefteq a.s$ if and only if $(a\,c) \cdot t \trianglelefteq (a\,c) \cdot s$. In the case of comparing two applications of the same commutative function $f^E(\overline{t})_n$ and $f^E(\overline{s})_m$ where $E \in \{C, AC\}$, the lexicographically ordered sequences of arguments $(t_{i_1} \trianglelefteq \ldots \trianglelefteq t_{i_n})$ and $(s_{j_1} \trianglelefteq \ldots \trianglelefteq s_{j_m})$ are being used for comparison, $\{i_1 \ldots, i_n\} \equiv \{1, \ldots, n\}, \{j_1 \ldots, j_m\} \equiv \{1, \ldots, m\}$. That is, $f^E(\overline{t})_n \trianglelefteq f^E(\overline{s})_m$ if and only if, either $n \leq m$ and $t_{i_1} = s_{j_1}, \ldots, t_{i_n} = s_{j_n}$, or there is a $k \leq \min(n, m)$ so that $t_{i_1} = s_{j_1}, \ldots, t_{i_{k-1}} = s_{j_{k-1}}, t_{i_k} \trianglelefteq s_{j_k}$ and $t_{i_k} \neq s_{j_k}$. If the function is not commutative, i.e., $E \in \{A, \emptyset\}$, then simply the sequences $\overline{t}_n$ and $\overline{s}_m$ are used for comparison.

Example 12. *For instance,* $a \trianglelefteq a \trianglelefteq c \trianglelefteq f(a, b) \trianglelefteq X \trianglelefteq (a\ b) \cdot X \trianglelefteq X \trianglelefteq b.X \trianglelefteq a.X \trianglelefteq a.b.f(a, X)$.

When using the branching heuristics, arguments of commutative function applications, i.e., with equational theory C or AC, are being kept ordered according to $\trianglelefteq$. For instance, $f^{AC}(b, a, c, a, b.b, g(a)) \triangleq f^{AC}(a.a, c, d, b, a, b, c, g(c))$ rewrites to $f^{AC}(a, a, b, c, g(a), b.b) \triangleq f^{AC}(a, b, b, c, c, d, g(c), a.a)$.

[†]Note that this resembles the behavior of the equivariance rule `Abs-E` in Figure 3.

Definition 6. *Let w_1 and w_2 be sequences of symbols (words). An* alignment *of w_1 and w_2 is a sequence of the form $a_1[i_1, j_1] \dots a_n[i_n, j_n]$, $n \geq 0$, such that*

- $0 < i_1 < \dots < i_n \leq |w_1|$ *and* $0 < j_1 < \dots < j_n \leq |w_2|$, *and*
- $a_k = w_1|_{i_k} = w_2|_{j_k}$ *for all* $1 \leq k \leq n$. *The symbol at position i of the word w is denoted by $w|_i$.*

A *rigidity function* $\mathcal{R}$ is a function that computes an alignment for two given words w_1 and w_2. $\mathcal{R}_{lcs}$ denotes a rigidity function that computes an alignment that is a longest common subsequence (lcs) of the given input words. If more than one lcs exists, it denotes an arbitrary one (non-deterministic). The function $\mathcal{H}$ returns the sequence of head symbols of a given sequence of terms. For instance, $\mathcal{H}(a, a, b, c, g(a), b.b) = (aabcg.)$.

Example 13. *Given the terms $f^{AC}(a, a, b, c, g(a), b.b)$ and $f^{AC}(a, b, b, c, c, d, g(c), a.a)$. $\mathcal{R}_{lcs}(\mathcal{H}(a, a, b, c, g(a), b.b), \mathcal{H}(a, b, b, c, c, d, g(c), a.a))$ computes the alignment $a[1, 1]b[3, 2]c[4, 4]g[5, 7].[6, 8]$, or a "similar" one.*

The branching heuristics are parametric by the rigidity function. However, the only one that we used in our experiments is $\mathcal{R}_{lcs}$, where an arbitrary lcs alignment is returned, if more than one exists. The heuristics do not change the result set of computed generalizations. This stands in contrast to other approaches like [16, 17, 18]. Our heuristics serve the purpose of selecting promising branches first. We archive that by rearranging the arguments of commutative function applications in accordance to the rigidity function, while applying (`DecC`) or (`DecAC`). Note that, both, the relation $\trianglelefteq$ and the alignments are based on the head symbols. Moreover, the sets $\mathcal{A}$, $\mathcal{X}$ and Σ are assumed to be pairwise disjoint.

Given $f^E(t_1, \dots, t_n) \triangleq f^E(s_1, \dots, s_m)$ and a rigidity function $\mathcal{R}$. When the heuristics are enabled and $E \in \{AC, C\}$, then $t_1 \trianglelefteq \dots \trianglelefteq t_n$ and $s_1 \trianglelefteq \dots \trianglelefteq s_m$. In that case the decomposition procedures works in the following manner:

1. Separate f^E from the arguments $(t_1, \dots, t_n)$ and $(s_1, \dots, s_m)$.
2. Compute $a_1[i_1, j_1] \dots a_k[i_k, j_k] = \mathcal{R}(\mathcal{H}(t_1, \dots, t_n), \mathcal{H}(s_1, \dots, s_m))$.
3. Arrange $(t_{i_1}, \dots, t_{i_k}, \overline{r}_1)$ and $(s_{j_1}, \dots, s_{j_k}, \overline{r}_2)$ where $\overline{r}_1$ and $\overline{r}_2$, respectively, denote the arguments from $(t_1, \dots, t_n)$ and $(s_1, \dots, s_m)$ that do not appear in $(t_{i_1}, \dots, t_{i_k})$ and $(s_{j_1}, \dots, s_{j_k})$.

4. Use f^E, $(t_{i_1}, \ldots, t_{i_k}, \overline{r}_1)$ and $(s_{j_1}, \ldots, s_{j_k}, \overline{r}_2)$ to create the branches according to the implemented strategy (Subsection 6.1).

Example 14. *Consider the flattened $\{a, b, c, d\}$-based terms $t = f^{AC}(b, a, c, a, b.b, g(a))$ and $s = f^{AC}(a.a, c, d, b, a, b, c, g(c))$ with the empty freshness context. Even though the terms are relatively small, repeatetly applying the rule (*`DecAC`*) in all possible ways produces a large amount of branches. The first decomposition already produces $(2^5 - 1) \times (2^7 - 1) \times 2 = 7874$ branches. To each of them, the rule (*`DecAC`*) is applicable again, et cetera. Observe that, any $\{a, b, c, d\}$-based lgg of s and t contains an application of the function g and the atom b as arguments under the application of f^{AC}. In an experiment we iterated the first million branches with heuristics disabled. There was not a single result where the function g and the atom b appeared as arguments under the application of f^{AC}.*[†] *That is, without heuristics, there is no $\{a, b, c, d\}$-based lgg of s and t among the first million generated branches. Now we illustrate that, with the heuristics enabled, the first branch already leads to a least general generalization. For the sake of readability, we focus on the set P used in the rules of $\mathtt{NAU}_E$. First, the arguments of the application of f^{AC} are being ordered:*

$$f^{AC}(a, a, b, c, g(a), b.b) \triangleq f^{AC}(a, b, b, c, c, d, g(c), a.a).$$

The alignment computed by $\mathcal{R}_{lcs}$ is $a[1, 1]b[3, 2]c[4, 4]g[5, 7].[6, 8]$ and leads to

$$a \triangleq a \text{ and } f^{AC}(a, b, c, g(a), b.b) \triangleq f^{AC}(b, b, c, c, d, g(c), a.a)$$

being the first branch, generated by (`DecAC`*). After repeatedly applying the decomposition procedure to the first generated branch we get*

$$a \triangleq a,\ b \triangleq b,\ c \triangleq c,\ g(a) \triangleq g(c),\ b.b \triangleq a.a,\ a \triangleq f^{AC}(b, c, d).$$

This results in the $\{a, b, c, d\}$-based lgg $\langle b\#X, d\#X, f^{AC}(a, b, c, g(X), a.a, Y)\rangle$ of s and t. To conclude the example, we were not able to find a single lgg of s and t without the branch selection heuristics, whereas with the heuristics enabled, the first branch emits an lgg.

There is no guaranteed upper bound on the number of branches needed to get a least general generalization, aside of the trivial one to iterate all

[†]The output was simply filtered using `grep 'g\[' | grep -E '((b,)|(b\]))'`

possibilities, even with the heuristics enabled. However, since the number of branches grow exponentially in the input size, there are cases where iterating all the possibilities becomes impossible. In such cases the heuristics can be used to get better results. Note that, even when considering only the first branch that is generated by $\texttt{NAU}_E$, it does not become tractable due to the equational reasoning in EQVM.

6.3. Tractable Generalization Computation with $\texttt{NAU}_E$

Since, by Theorem 6, $\texttt{NAU}_\emptyset$ is tractable, the two sources of intractability of $\texttt{NAU}_E$ can easily be identified as:

1. Branching in the equational decomposition rules.

2. The sub-algorithm EQVM in the merge rule, that must consider $\approx_E$.

A term-in-context $\langle \nabla, r \rangle$ is *linear* iff each variable from $\mathcal{X}$ occurs at most once in r. That is, no variable is allowed to be used twice in r. A *linear E-generalization* (possibly based on a finite set of atoms) of two terms-in-context is a generalization of them that is linear. Other notions, for instance, *linear* $\texttt{lgg}_E$ are naturally adapted to the linear case.

A complete set of linear E-generalization can be computed by simply disabling the merge rule of $\texttt{NAU}_E$. This comes from the fact that all the other rules introduce fresh generalization variables that are only used once. By doing so, one source of intractability is being eliminated.

Fortunately, the heuristics described in the former subsection have relatively low computational cost. That is, neither the ordering $\trianglelefteq$ nor the alignment computation with $\mathcal{R}_{lcs}$ introduce an exponential blow-up. Therefore, considering a few branches that emit promising linear E-generalizations can be done in polynomial time w.r.t. the input size.

Corollary 1. *Given a fixed $k \geq 0$, a finite set of atoms A, two A-based terms t, s, an A-based freshness context ∇, and a rigidity function that runs in polynomial time. Let the input size be n. Computing $\mathcal{O}(n^k)$ approximations of linear A-based equational lggs of $\langle \nabla, t \rangle$ and $\langle \nabla, s \rangle$ is tractable.*

It is worth mentioning that all the heuristics discussed in this section can be separately enabled or disabled in our implementation.

	Syntactic	A, C, AC
Original nominal theory	Nullary [7]	Nullary [7]
Nominal theory with finite set of atoms	Unitary [7]	**Finitary**
Nominal theory with atom variables	Finitary [25]	Finitary [26]
Permissive nominal theory	Nullary [27]	Nullary [27]
Permissive with extended permission sets	Unitary [27]	?
Higher-order pattern	Unitary [3]	Finitary [4]

Figure 5: Overview of generalization types in related theories.

7. Related Work

Anti-unification in first-order languages modulo equational theories has been studied for decades [1, 15, 19]. More recent work has extended this line of research to accommodate increasingly complex theories [20, 21, 22]. However, comparatively less attention has been given to anti-unification in languages that feature binding constructs, freshness conditions, and reasoning modulo equational theories, where additional challenges such as α-equivalence and name permutation arise.

Existing higher-order (HO) approaches to anti-unification [3, 4] typically operate on the $\beta\eta$-normal forms of higher-order patterns. Higher-order patterns are a restricted class of λ-terms in which free variables may only be applied to a sequence of distinct bound variables. Introduced by [23], this restriction makes unification decidable by reducing it to a process similar to first-order unification, thus avoiding the undecidability of general higher-order unification [24].

These HO approaches generally assume—at the meta-level—that canonical representatives have been chosen from each α-equivalence class. This assumption allows the frameworks to bypass explicit reasoning about variable renaming and freshness constraints. However, these assumptions are deeply tied to the simply typed λ-calculus and may not generalize to other binding frameworks, where binders originate from different syntactic or semantic structures. Moreover, they present obstacles for formalization and mechanization, particularly in systems where precise control over binding is essential. See Pitts [12] for a detailed discussion on the formalization of inductive reasoning, with particular attention to assumptions made in the meta-language.

We observe that there is a connection between nominal anti-unification problems and higher-order pattern anti-unification problems. This is not

surprising, given that nominal unification can be reduced to higher-order pattern unification, as shown in [28]. Several of our examples are nominal adaptations of examples from [4], which were originally formulated in the higher-order pattern setting. The generalizations that we obtained related with the generalizations obtained in [4]. The main difference is that we do not need to reason modulo β- and η-equivalences, and freshness conditions are explicitly handled by the nominal language.

Figure 5 provides an overview of existing results on anti-unification problems within the nominal framework and the higher-order pattern approach. Within the nominal approach, the problem was first studied in [7], where it was shown that, unless the set of atoms permitted in generalizations is restricted to be finite, a least general generalization (lgg) of two terms-in-context may not exist (see Remark 1). In the equational setting, this implies that, without such a restriction, a minimal complete set of E-generalizations might not exist. An alternative for solving nominal anti-unification without a fixed atom set was investigated in [25] considering a semantic approach to generalizations. The problem becomes finitary, but no without equational theories were considered. An extension of the nominal anti-unification problem with atom variables and equational theories A,C and AC is under work [26], as expected the problem is finitary. Preliminary work has been done in anti-unification in the permissive nominal language [29] without equational theories.

Applications of anti-unification span a wide range of domains. It has been employed in recursion scheme generalization and clone detection [30, 31], counterexample-based learning [32], higher-order term indexing [33], and automated library learning [2]. In inductive logic programming (ILP), early approaches used anti-unification to derive the most specific clause that entails a given example [34, 35]. More recent work applies anti-unification to reduce the search space in ILP systems [36, 37]. Our results are likely to find applications in the learning of higher-order programs, adapting the methodology proposed in [38] to the nominal framework.

8. Conclusion

We investigated the extension of the nominal anti-unification problem with the equational theories A, C and AC. Since, in general, the nominal anti-unification problem is known to be nullary, we follow the approach from [7] and restrict the set of considered atoms to be finite. In this setting the

nominal anti-unification problem modulo A, C and AC becomes finitary. We extended the algorithm that solves the nominal anti-unification problem in the empty theory by adding specific rules to deal with equational theories. Moreover, the subalgorithm that is used to solve the equivariance problem has been adapted.

Unfortunately, equational theories lead to intractability of both, the anti-unification algorithm and the equivariance algorithm. For that reason, we studied heuristics that can be used to improve efficiency in practice. By considering only linear generalizations, it becomes even possible to compute some promising generalizations of two arbitrary input terms-in-context in polynomial time.

It is known that there is a reduction from nominal unification to higher-order pattern unification and it would be interesting to know whether there is a simple reduction from our setting to the setting of equational higher-order pattern anti-unification from [18]. It would also be interesting to check how the nominal anti-unification algorithm deals with recursive let as in [39].

References

[1] G. D. Plotkin, A note on inductive generalization, Machine Intell. 5 (1) (1970) 153–163.

[2] D. Cao, R. Kunkel, C. Nandi, M. Willsey, Z. Tatlock, N. Polikarpova, babble: Learning better abstractions with e-graphs and anti-unification, Proc. ACM Program. Lang. 7 (POPL) (2023) 396–424. doi:10.1145/3571207.
URL https://doi.org/10.1145/3571207

[3] A. Baumgartner, T. Kutsia, J. Levy, M. Villaret, Higher-order pattern anti-unification in linear time, J. Autom. Reasoning 58 (2) (2017) 293–310. doi:10.1007/s10817-016-9383-3.
URL https://doi.org/10.1007/s10817-016-9383-3

[4] D. M. Cerna, T. Kutsia, Higher-order pattern generalization modulo equational theories, Math. Struct. Comput. Sci. 30 (6) (2020) 627–663. doi:10.1017/S0960129520000110.
URL https://doi.org/10.1017/S0960129520000110

[5] D. M. Cerna, T. Kutsia, Anti-unification and generalization: A survey, in: Proceedings of the Thirty-Second International Joint Conference on

Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China, ijcai.org, 2023, pp. 6563–6573. doi:10.24963/IJCAI.2023/736. URL https://doi.org/10.24963/ijcai.2023/736

[6] M. J. Gabbay, A. Mathijssen, One-and-a-halfth-order logic, in: Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming, PPDP '06, Association for Computing Machinery, New York, NY, USA, 2006, p. 189–200. doi:10.1145/1140335.1140359.

[7] A. Baumgartner, T. Kutsia, J. Levy, M. Villaret, Nominal anti-unification, in: M. Fernández (Ed.), 26th International Conference on Rewriting Techniques and Applications, RTA 2015, June 29 to July 1, Warsaw, Poland, Vol. 36 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2015, pp. 57–73. doi:10.4230/LIPIcs.RTA.2015.57. URL http://dx.doi.org/10.4230/LIPIcs.RTA.2015.57

[8] A. Baumgartner, D. Nantes-Sobrinho, A, c, and AC nominal anti-unification, in: T. Kutsia, A. M. Marshall (Eds.), Proceedings of the 34th International Workshop on Unification, UNIF 2020, Linz, Austria, June 29, 2020, 2020, pp. 5:1–5:6.
URL http://www3.risc.jku.at/publications/download/risc_6129/proceedings-UNIF2020.pdf#page=27

[9] M. Fernández, M. Gabbay, Nominal rewriting, Inf. Comput. 205 (6) (2007) 917–965. doi:10.1016/j.ic.2006.12.002.
URL https://doi.org/10.1016/j.ic.2006.12.002

[10] A. M. Pitts, Nominal Sets: Names and Symmetry in Computer Science, Cambridge University Press, 2013.

[11] M. Ayala-Rincón, W. de Carvalho Segundo, M. Fernández, D. Nantes-Sobrinho, A. C. R. Oliveira, A formalisation of nominal α-equivalence with A, C, and AC function symbols, Theor. Comput. Sci. 781 (2019) 3–23. doi:10.1016/j.tcs.2019.02.020.
URL https://doi.org/10.1016/j.tcs.2019.02.020

[12] A. M. Pitts, Nominal logic, a first order theory of names and binding, Inf. Comput. 186 (2) (2003) 165–193. doi:10.1016/S0890-5401(03)

00138-X.
URL https://doi.org/10.1016/S0890-5401(03)00138-X

[13] A. Baumgartner, T. Kutsia, J. Levy, M. Villaret, Nominal Anti-Unification, Tech. Rep. 15-03, RISC (April 2015).
URL https://www3.risc.jku.at/publications/download/risc_5145/nau-tr.pdf

[14] M. Alpuente, S. Escobar, J. Espert, J. Meseguer, ACUOS: A system for modular ACU generalization with subtyping and inheritance, in: Fermé and Leite [40], pp. 573–581.

[15] M. Alpuente, S. Escobar, J. Espert, J. Meseguer, A modular order-sorted equational generalization algorithm, Inf. Comput. 235 (2014) 98–136.

[16] T. Kutsia, J. Levy, M. Villaret, Anti-unification for unranked terms and hedges, J. Autom. Reasoning 52 (2) (2014) 155–190.

[17] A. Baumgartner, T. Kutsia, Unranked second-order anti-unification, Inf. Comput. 255 (2017) 262–286.

[18] D. M. Cerna, T. Kutsia, Higher-order equational pattern anti-unification, in: H. Kirchner (Ed.), 3rd International Conference on Formal Structures for Computation and Deduction, FSCD 2018, July 9-12, 2018, Oxford, UK, Vol. 108 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, pp. 12:1–12:17. doi:10.4230/LIPIcs.FSCD.2018.12.
URL https://doi.org/10.4230/LIPIcs.FSCD.2018.12

[19] F. Baader, Unification, weak unification, upper bound, lower bound, and generalization problems, in: R. V. Book (Ed.), Rewriting Techniques and Applications, 4th International Conference, RTA-91, Como, Italy, April 10-12, 1991, Proceedings, Vol. 488 of Lecture Notes in Computer Science, Springer, 1991, pp. 86–97. doi:10.1007/3-540-53904-2_88.
URL https://doi.org/10.1007/3-540-53904-2_88

[20] D. M. Cerna, T. Kutsia, Idempotent anti-unification, ACM Trans. Comput. Log. 21 (2) (2020) 10:1–10:32. doi:10.1145/3359060.
URL https://doi.org/10.1145/3359060

[21] D. M. Cerna, Anti-unification and the theory of semirings, Theor. Comput. Sci. 848 (2020) 133–139. doi:10.1016/J.TCS.2020.10.020.
URL https://doi.org/10.1016/j.tcs.2020.10.020

[22] M. Ayala-Rincón, D. M. Cerna, A. F. G. Barragan, T. Kutsia, Equational anti-unification over absorption theories, in: C. Benzmüller, M. J. H. Heule, R. A. Schmidt (Eds.), Automated Reasoning - 12th International Joint Conference, IJCAR 2024, Nancy, France, July 3-6, 2024, Proceedings, Part II, Vol. 14740 of Lecture Notes in Computer Science, Springer, 2024, pp. 317–337. doi:10.1007/978-3-031-63501-4_17.
URL https://doi.org/10.1007/978-3-031-63501-4_17

[23] D. Miller, A logic programming language with lambda-abstraction, function variables, and simple unification, J. Log. Comput. 1 (4) (1991) 497–536. doi:10.1093/logcom/1.4.497.
URL https://doi.org/10.1093/logcom/1.4.497

[24] G. P. Huet, The undecidability of unification in third order logic, Inf. Control. 22 (3) (1973) 257–267. doi:10.1016/S0019-9958(73)90301-X.
URL https://doi.org/10.1016/S0019-9958(73)90301-X

[25] M. Schmidt-Schauß, D. Nantes-Sobrinho, Nominal anti-unification with atom-variables, in: A. P. Felty (Ed.), 7th International Conference on Formal Structures for Computation and Deduction, FSCD 2022, August 2-5, 2022, Haifa, Israel, Vol. 228 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, pp. 7:1–7:22. doi:10.4230/LIPICS.FSCD.2022.7.
URL https://doi.org/10.4230/LIPIcs.FSCD.2022.7

[26] D. Nantes-Sobrinho, M. Schmidt-Schauss, A. Baumgartner, T. Kutsia, Generalization problems with atom-variables in languages with binders and equational theories (2025). arXiv:2502.19306.
URL https://arxiv.org/abs/2502.19306

[27] A. Baumgartner, Towards a well-founded $\preceq$ relation for permissive nominal terms, in: S. Escobar, O. F. Gil (Eds.), The 38th International Workshop on Unification – UNIF 2024 – Workshop Informal Proceedings, 2024, pp. 20–27.

URL https://lat.inf.tu-dresden.de/unif2024/informal-proceedings-UNIF2024.pdf

[28] J. Levy, M. Villaret, Nominal unification from a higher-order perspective, ACM Trans. Comput. Log. 13 (2) (2012) 10:1–10:31. doi:10.1145/2159531.2159532.
URL https://doi.org/10.1145/2159531.2159532

[29] A. Baumgartner, Towards a well-founded $\preceq$ relation for permissive nominal terms, in: S. Escobar, O. F. Gil (Eds.), The 38th International Workshop on Unification – UNIF 2024 – Workshop Informal Proceedings, 2024, pp. 20–27.
URL https://lat.inf.tu-dresden.de/unif2024/informal-proceedings-UNIF2024.pdf

[30] A. D. Barwell, C. Brown, K. Hammond, Finding parallel functional pearls: Automatic parallel recursion scheme detection in Haskell functions via anti-unification, Future Generation Comp. Syst. 79 (2018) 669–686.

[31] A. Baumgartner, T. Kutsia, A library of anti-unification algorithms, in: Fermé and Leite [40], pp. 543–557. doi:10.1007/978-3-319-11558-0_38.
URL https://doi.org/10.1007/978-3-319-11558-0_38

[32] J. Lassez, K. Marriott, Explicit representation of terms defined by counter examples, J. Autom. Reason. 3 (3) (1987) 301–317. doi:10.1007/BF00243794.
URL https://doi.org/10.1007/BF00243794

[33] B. Pientka, Higher-order term indexing using substitution trees, ACM Trans. Comput. Log. 11 (1) (2009) 6:1–6:40. doi:10.1145/1614431.1614437.
URL https://doi.org/10.1145/1614431.1614437

[34] S. H. Muggleton, Inverse entailment and progol, New Gener. Comput. 13 (3&4) (1995) 245–286. doi:10.1007/BF03037227.
URL https://doi.org/10.1007/BF03037227

[35] S. H. Muggleton, J. C. A. Santos, A. Tamaddoni-Nezhad, Progolem: A system based on relative minimal generalisation, in: L. D. Raedt (Ed.),

Inductive Logic Programming, 19th International Conference, ILP 2009, Leuven, Belgium, July 02-04, 2009. Revised Papers, Vol. 5989 of Lecture Notes in Computer Science, Springer, 2009, pp. 131–148. doi:10.1007/978-3-642-13840-9_13.
URL https://doi.org/10.1007/978-3-642-13840-9_13

[36] A. Cropper, R. Morel, Learning programs by learning from failures, Mach. Learn. 110 (4) (2021) 801–856. doi:10.1007/S10994-020-05934-Z.
URL https://doi.org/10.1007/s10994-020-05934-z

[37] A. Cropper, S. Dumancic, R. Evans, S. H. Muggleton, Inductive logic programming at 30, Mach. Learn. 111 (1) (2022) 147–172. doi:10.1007/S10994-021-06089-1.
URL https://doi.org/10.1007/s10994-021-06089-1

[38] A. Cropper, R. Morel, S. H. Muggleton, Learning higher-order logic programs, Mach. Learn. 109 (7) (2020) 1289–1322. doi:10.1007/S10994-019-05862-7.
URL https://doi.org/10.1007/s10994-019-05862-7

[39] M. Schmidt-Schauß, T. Kutsia, J. Levy, M. Villaret, Nominal unification of higher order expressions with recursive let, in: M. V. Hermenegildo, P. López-García (Eds.), Logic-Based Program Synthesis and Transformation - 26th International Symposium, LOPSTR 2016, Edinburgh, UK, September 6-8, 2016, Revised Selected Papers, Vol. 10184 of Lecture Notes in Computer Science, Springer, 2016, pp. 328–344. doi:10.1007/978-3-319-63139-4_19.
URL https://doi.org/10.1007/978-3-319-63139-4_19

[40] E. Fermé, J. Leite (Eds.), Logics in Artificial Intelligence - 14th European Conference, JELIA 2014, Funchal, Madeira, Portugal, September 24-26, 2014. Proceedings, Vol. 8761 of Lecture Notes in Computer Science, Springer, 2014.